CS23332:Database Management Systems

UNIT II

PL/SQL, DATA MODEL AND QUERY PROCESSING

Syllabus

UNITII	PL/SQL, DATA MODEL AND QUERY PROCESSING	9
PL/SQL: Functions, Procedures, Triggers, Cursors –Dynamic SQL–Relational Algebra-Tuple Relational calculus- Domain Relational Calculus– Entity Relationship Model – Constraints -Entity Relationship Diagram - Design Issues of ER Model – Extended ER Features – Mapping ER Model to Relational Model– Query Processing – Heuristics for Query Optimization.		

PL/SQL- Introduction

- Procedural Language Extension of SQL.
- An extension to SQL with design features of programming languages(Procedure and Object Oriented).

PL SQL Engine

- Oracle uses PL SQL Engine to processes the SQL statements.

- PL SQL code can be stored in the client system (client side) or in the database(server side).

Advantages of PL SQL

- It is completely portable and high performance transaction processing language that offers following advantages:

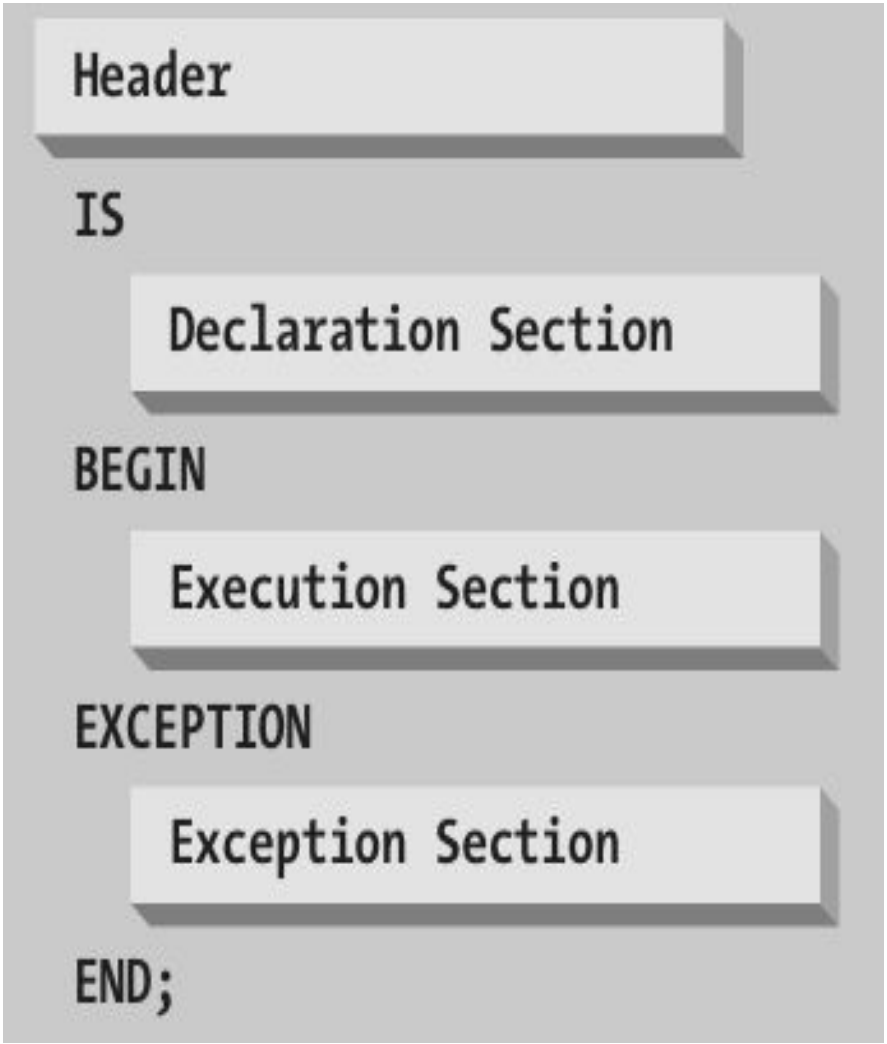
- Block Structure
- Procedural Language capability
- Better Performance
- Error Handling
- Portability
- Security

PL/SQL Block Structure

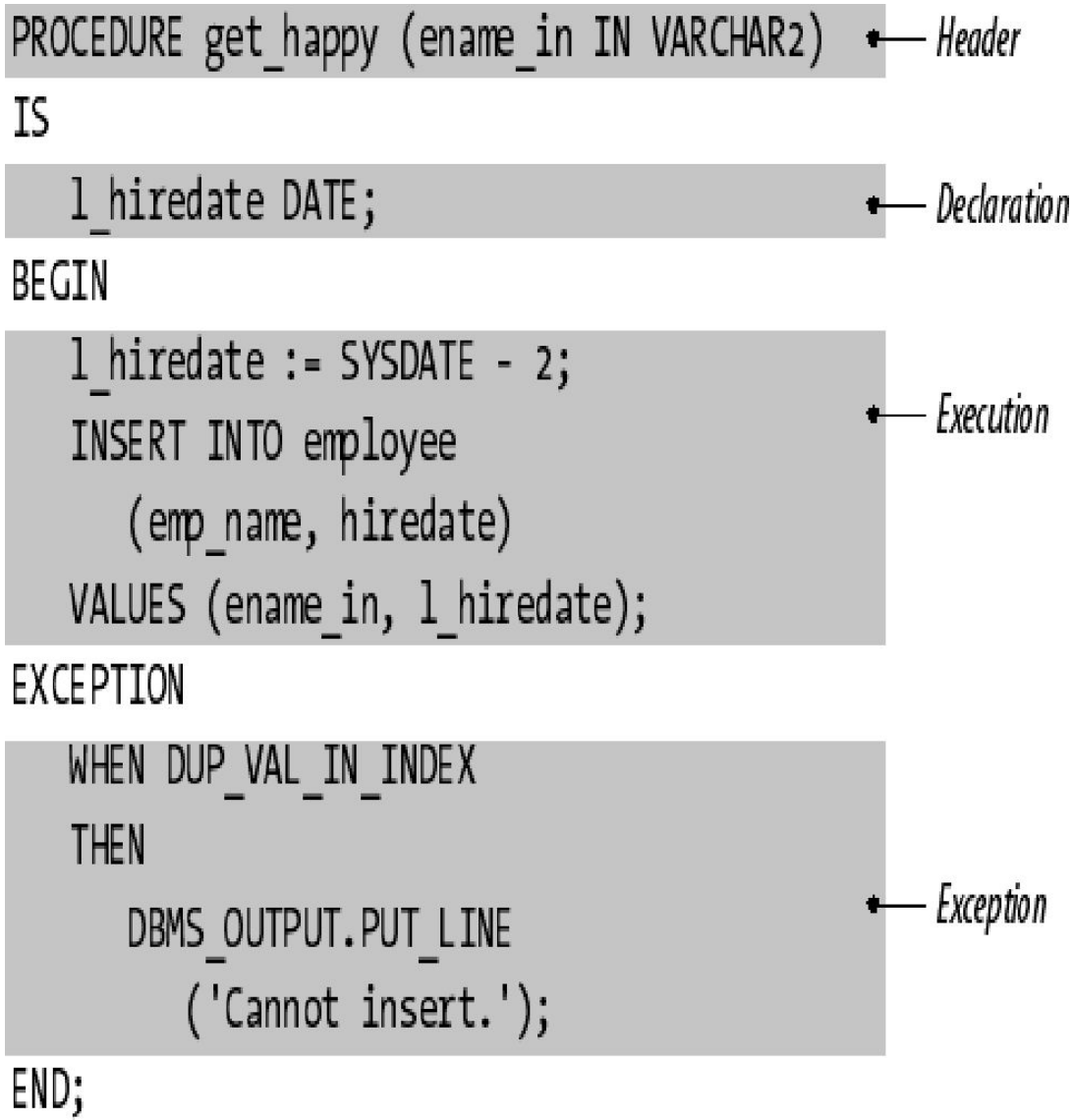
- A PL/SQL block has up to four different sections, only one of which is mandatory:

- **Header**-Used only for named blocks. The header determines the way the named block or program must be called. Optional.
- **Declaration section**-Identifies variables, cursors, and subblocks that are referenced in the execution and exception sections. Optional.
- **Execution section**-Statements the PL/SQL runtime engine will execute at runtime. Mandatory.
- **Exception section**-Handles exceptions to normal processing (warnings and error conditions). Optional.

PL/SQL Block Structure



PL/SQL Block Structure-Example



PL/SQL Language Constructs

- SQL supports constructs that gives it almost all the power of a general-purpose programming language.
  - Warning: most database systems implement their own variant of the standard syntax below.
- Compound statement: **begin** … **end,**
  - May contain multiple SQL statements between **begin** and **end**.
  - Local variables can be declared within a compound statements
- While and repeat statements:
  - **while** boolean expression **do**

    sequence of statements ;

    end while
- **Loop**

  sequence of statements ;

  Exit when boolean expression;

  end Loop;

PL/SQL Conditional Statement

- Conditional statements (**if-then-else**)

if *boolean expression*

then *statement or compound statement*

elseif *boolean expression*

then *statement or compound statement*

else *statement or compound statement*

end if

PL/SQL – For Loop

- **For** loop
  - Permits iteration over all results of a query
- Example: Find the budget of all departments

```
declare n integer default 0;
for r as
    select budget from department    where dept_name =
'Music'
do
   set n = n + r.budget
end for
```

PL SQL Function

- The PL/SQL Function is very similar to PL/SQL Procedure.
- The main difference between procedure and a function is, a function must always return a value, and on the other hand a procedure may or may not return a value.
- Except this, all the other things of PL/SQL procedure are true for PL/SQL function too.

Syntax: Create a Function

CREATE [OR REPLACE] **FUNCTION** function_name [parameters]
[(parameter_name [IN | **OUT** | IN **OUT**] type [, ...])]
RETURN return_datatype
{**IS** | **AS**}
BEGIN
  < function_body >
END [function_name];

- **Function_name:** specifies the name of the function.
- **[OR REPLACE]** option allows modifying an existing function.
- The **optional parameter list** contains name, mode and types of the parameters.
- **IN** represents that value will be passed from outside and OUT represents that this parameter will be used to return a value outside of the procedure.

PL/SQL Function Example

- Define a function that, given the name of a department, returns the count of the number of instructors in that department.
- **create function** *dept_count* (*dept_name* **varchar**(20))
  returns integer
  begin
  declare *d_count* **integer;**
  select count (*) **into** *d_count*
  from *instructor*
  where *instructor.dept_name = dept_name*
  return *d_count;*
  end
- The function *dept*_count can be used to find the department names and budget of all departments with more that 12 instructors.
- **select** *dept_name, budget*
  from *department*
  where *dept*_count (*dept_name*) > 12

PL SQL Procedures

- A PL SQL procedure is a group of PL/SQL block which performs one or more specific tasks.

- It is a module performing one or more actions, it does not return any value.

Creating a Procedure

CREATE [OR REPLACE] **PROCEDURE** procedure_name
  [(parameter name [IN | OUT] type[,…])]
IS | AS
  [declaration_section]
BEGIN
  procedure body
END [procedure_name];

Where

- procedure_name specifies name of the procedure
- [OR REPLACE] option allows modifying existing procedure
- **IN** represents that value will be passed from outside and **OUT** represents that this procedure will be used to return a value outside of the procedure.
- Procedure body contains the executable parts
- **AS** keyword is used instead of IS keyword for creating a standalone procedure

PL SQL Procedure-Example

```
create or replace procedure "INSERTUSER"
(id IN NUMBER,
name IN VARCHAR2)
is
begin
insert into user values(id,name);
end;
```

Procedure - Example

- CREATE OR REPLACE PROCEDURE query_emp (p_id IN employees.employee_id%TYPE,
  p_name OUT employees.last_name%TYPE,
  p_salary OUT employees.salary%TYPE)
  IS
  BEGIN
  SELECT last_name, salary INTO p_name, p_salary FROM employees
  WHERE employee_id = p_id; END query_emp;

/

Procedure created.

Executing a Standalone Procedure

- A Standalone Procedure can be called in two ways:

❑ Using the EXECUTE keyword

❑ Calling the name of the procedure from a PL / SQL block

- Deleting a Standalone Procedure:

DROP PROCEDURE procedure_name

Execution of PL/SQL Procedure

- -- Execute the procedure

```
DECLARE v_emp_name employees.last_name%TYPE;
v_emp_sal employees.salary%TYPE;
BEGIN
query_emp (171, v_emp_name, v_emp_sal);
DBMS_OUTPUT.PUT_LINE (v_emp_name || ' earns ' ||
TO_CHAR (v_emp_sal, '$999,999.00'));
END;
```

Smith earns $7,400.00

PL/SQL procedure successfully completed.

Example: Procedures

- Registers student after ensuring classroom capacity is not exceeded
  - Returns 0 on success and -1 if capacity is exceeded
- Signaling of exception conditions, and declaring handlers for exceptions
- **declare** *out_of_classroom_seats* **condition**
  declare exit handler for *out_of_classroom_seats*
  begin
  …
  end
- The statements between the **begin** and the **end** can raise an exception by executing "**signal** *out_of_classroom_seats*"
- The handler says that if the condition arises he action to be taken is to exit the enclosing the **begin end** statement.

External Language Routines

- SQL allows us to define functions in a programming language such as Java, C#, C or C++.
  - Can be more efficient than functions defined in SQL, and computations that cannot be carried out in SQL\can be executed by these functions.
- Declaring external language procedures and functions
- **create procedure** dept_count_proc(**in** *dept_name* **varchar**(20), **out** count **integer**)
  language C
  external name '/usr/avi/bin/dept_count_proc‘

create function dept_count(*dept_name* **varchar**(20))
returns integer
language C
external name '/usr/avi/bin/dept_count'

External Language Routines Benefits and Drawbacks

- Benefits of external language functions/procedures:
  - more efficient for many operations, and more expressive power.
- Drawbacks
  - Code to implement function may need to be loaded into database system and executed in the database system's address space.
    - risk of accidental corruption of database structures
    - security risk, allowing users access to unauthorized data
  - There are alternatives, which give good security at the cost of potentially worse performance.
  - Direct execution in the database system's space is used when efficiency is more important than security.

Triggers

- A **trigger** is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
  - Specify the conditions under which the trigger is to be executed.
  - Specify the actions to be taken when the trigger executes.
- Triggers introduced to SQL standard in SQL:1999, but supported even earlier using non-standard syntax by most databases.
  - Syntax illustrated here may not work exactly on your database system; check the system manuals

Triggering Events and Actions in SQL

- Triggering event can be **insert**, **delete** or **update**
- Triggers on update can be restricted to specific attributes
  - For example, **after update of** *takes* **on** *grade*
- Values of attributes before and after an update can be referenced
  - **referencing old row as** : for deletes and updates
  - **referencing new row as** : for inserts and updates
- Triggers can be activated before an event, which can serve as extra constraints. For example, convert blank grades to null.
- **create trigger** *setnull_trigger* **before update of** *takes*
  referencing new row as *nrow*
  for each row
  when (*nrow.grade* = ' ')
  begin atomic
  set *nrow.grade* = **null;**
  end;

Trigger Events illustration

```
create trigger credits_earned after update of takes on
(grade)
referencing new row as nrow
referencing old row as orow
for each row
when nrow.grade <> 'F' and nrow.grade is not null
   and (orow.grade = 'F' or orow.grade is null)
begin atomic
   update student
   set tot_cred= tot_cred +
        (select credits
         from course
         where course.course_id= nrow.course_id)
   where student.id = nrow.id;
end;
```

Statement Level Triggers

- Instead of executing a separate action for each affected row, a single action can be executed for all rows affected by a transaction
  - Use **for each statement** instead of **for each row**
  - Use **referencing old table** or **referencing new table** to refer to temporary tables (called ***transition tables***) containing the affected rows
  - Can be more efficient when dealing with SQL statements that update a large number of rows

When not to use Triggers

- Triggers were used earlier for tasks such as
  - Maintaining summary data (e.g., total salary of each department)
  - Replicating databases by recording changes to special relations (called **change** or **delta** relations) and having a separate process that applies the changes over to a replica
- There are better ways of doing these now:
  - Databases today provide built in materialized view facilities to maintain summary data
  - Databases provide built-in support for replication
- Encapsulation facilities can be used instead of triggers in many cases
  - Define methods to update fields
  - Carry out actions as part of the update methods instead of through a trigger

When not to use Triggers(cont.)

- Risk of unintended execution of triggers, for example, when
  - Loading data from a backup copy
  - Replicating updates at a remote site
  - Trigger execution can be disabled before such actions.

- Other risks with triggers:
  - Error leading to failure of critical transactions that set off the trigger
  - Cascading execution

PL SQL Cursor

- When an SQL statement is processed, Oracle creates a memory area known as context area. A cursor is a pointer to this context area. It contains all information needed for processing the statement.

- In PL/SQL, the context area is controlled by Cursor. A cursor contains information on a select statement and the rows of data accessed by it.

- A cursor is used to referred to a program to fetch and process the rows returned by the SQL statement, one at a time. There are two types of cursors:

Types of Cursor

- There are two types of Cursor:

❑ Implicit Cursor

❑ Explicit Cursor

Implicit Cursor

- The implicit cursors are automatically generated by Oracle while an SQL statement is executed.

- These are created by default to process the statements when DML statements like INSERT, UPDATE, DELETE etc. are executed.

- Orcale provides some attributes known as Implicit cursor's attributes to check the status of DML operations. Some of them are: %FOUND, %NOTFOUND, %ROWCOUNT and %ISOPEN.

Implicit Cursor- Example

```
DECLARE
  total_rows number(2);
BEGIN
  UPDATE  customers
  SET salary = salary + 5000;
  IF sql%notfound THEN
    dbms_output.put_line('no customers updated');
  ELSIF sql%found THEN
    total_rows := sql%rowcount;
    dbms_output.put_line( total_rows || ' customers updated ');
  END IF;
END;
```

Output:

6 customers updated

PL/SQL procedure successfully completed.

Explicit Cursor

- The Explicit cursors are defined by the programmers to gain more control over the context area.

- These cursors should be defined in the declaration section of the PL/SQL block. It is created on a SELECT statement which returns more than one row.

- Syntax of Explicit cursor:

  CURSOR cursor_name **IS** select_statement;;

Steps: Working with Explicit Cursor

- Declare the cursor to initialize in the memory.
- Open the cursor to allocate memory.
- Fetch the cursor to retrieve data.
- Close the cursor to release allocated memory.

Working with Explicit Cursor(Cont.)

1**) Declare the cursor:**

It defines the cursor with a name and the associated SELECT statement.

- **CURSOR name IS**

  SELECT statement;

2**) Open the cursor:**

It is used to allocate memory for the cursor and make it easy to fetch the rows returned by the SQL statements into it.

Syntax for cursor open:

- **OPEN** cursor_name;

Working with Explicit Cursor(Cont.)

3) **Fetch the cursor:**

- It is used to access one row at a time. You can fetch rows from the above-opened cursor as follows:

FETCH cursor_name **INTO** variable_list;

4) Close the cursor:

- It is used to release the allocated memory. The following syntax is used to close the above-opened cursors.

Close cursor_name;

Explicit Cursor- Example

```
DECLARE
   c_id customers.id%type;
   c_name customers.name%type;
   c_addr customers.address%type;
   CURSOR c_customers is
      SELECT id, name, address FROM customers;
BEGIN
   OPEN c_customers;
   LOOP
      FETCH c_customers into c_id, c_name, c_addr;
      EXIT WHEN c_customers%notfound;
      dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
   END LOOP;
   CLOSE c_customers;
END;
/
```

Embedded SQL

- The SQL standard defines embeddings of SQL in a variety of programming languages such as C, C++, Java, Fortran, and PL/1,
- A language to which SQL queries are embedded is referred to as a **host language**, and the SQL structures permitted in the host language comprise *embedded* SQL.
- The basic form of these languages follows that of the System R embedding of SQL into PL/1.
- **EXEC SQL** statement is used in the host language to identify embedded SQL request to the preprocessor
- EXEC SQL <embedded SQL statement >;
- Note: this varies by language:
- In some languages, like COBOL, the semicolon is replaced with END-EXEC
- In Java embedding uses # SQL { …. };

Embedded SQL

- Before executing any SQL statements, the program must first connect to the database. This is done using:
- EXEC-SQL **connect to** *server* **user** *user-name* **using** *password*;
- Here, *server* identifies the server to which a connection is to be established.
- Variables of the host language can be used within embedded SQL statements. They are preceded by a colon (:) to distinguish from SQL variables (e.g., :*credit_amount)*
- Variables used as above must be declared within DECLARE section, as illustrated below. The syntax for declaring the variables, however, follows the usual host language syntax.

```
EXEC-SQL BEGIN DECLARE SECTION}
    int credit-amount ;
EXEC-SQL END DECLARE SECTION;
```

Embedded SQL(Contd.)

- To write an embedded SQL query, we use the

declare *c* **cursor for <SQL query>**

statement. The variable *c* is used to identify the query

- Example:
  - From within a host language, find the ID and name of students who have completed more than the number of credits stored in variable credit_amount in the host langue
  - Specify the query in SQL as follows:

EXEC SQL

declare *c* **cursor for**
select *ID, name* **from** *student*
where tot_cred > *:credit_amount*

END_EXEC

Embedded SQL(Cont.)

- The **open** statement for our example is as follows:

  EXEC SQL **open** *c* ;

- This statement causes the database system to execute the query and to save the results within a temporary relation. The query uses the value of the host-language variable *credit-amount* at the time the **open** statement is executed.
- The fetch statement causes the values of one tuple in the query result to be placed on host language variables.
- EXEC SQL **fetch** *c* **into** :*si, :sn* END_EXEC

Repeated calls to fetch get successive tuples in the query result.

Embedded SQL(Cont.)

- A variable called SQLSTATE in the SQL communication area (SQLCA) gets set to '02000' to indicate no more data is available
- The **close** statement causes the database system to delete the temporary relation that holds the result of the query.
- EXEC SQL **close** *c* ;
- Note: above details vary with language. For example, the Java embedding defines Java iterators to step through result tuples.

Dynamic SQL

- Dynamic SQL involves the creation and execution of SQL statements at runtime.
- It allows developers to generate SQL statements dynamically based on runtime conditions or user input.
- Because of its adaptability, dynamic SQL is a good choice when the SQL statements need to change in response to evolving needs or user inputs.
- Dynamic SQL queries are built at execution time so the system chooses how to access the database and conduct the SQL queries. Performance could be affected as a result of this lack of preparation because the system must create an execution plan on the spot.
- However it provides outstanding adaptability and versatility.

Steps to use Dynamic SQL

Step 1: Declare two variables

- DECLARE @var1 NVARCHAR(MAX), @var2 NVARCHAR(MAX);

Step 2: Set the value of the first variable as table_name

- SET @var1 = N'table_name';

Step 3: Select statement is added to table_name to create dynamic SQL

- SET @var2= N'SELECT * FROM ' + @var1;

Step 4: Use the second variable to run the sp_executesql

- EXEC sp_executesql @var2;

Dynamic SQL- Example

- This example will show how the data of a table named 'student' is selected using a dynamic process.
- **Input Table**

Student

Id	Name	City
1	Monu	Gonda
2	Aman	Lucknow
3	Naman	Dehradun

Dynamic SQL- Example(Cont.)

```
DECLARE @var1 NVARCHAR(100),@var2 NVARCHAR(120);

SET @var1 = N'student';

SET @var2 = N'SELECT * FROM' + @var1;

EXEC sp_executesql @var2;
```

Dynamic SQL- Example(Cont.)

- Output

Student

Id	Name	City
1	Monu	Gonda
2	Aman	Lucknow
3	Naman	Dehradun

Advantages of Dynamic SQL

- **Flexibility:** Dynamic SQL provides unparalleled flexibility as it allows developers to build SQL statements dynamically based on runtime conditions or user input. This flexibility enables developers to create dynamic queries, adapt to changing requirements, and handle complex scenarios.
- **Conditional Queries:** Dynamic SQL is particularly useful when dealing with conditional queries. By building SQL statements dynamically, developers can incorporate conditions into the queries, such as dynamic WHERE clauses or varying column selections based on runtime conditions.
- **Table and Column Name Flexibility:** Dynamic SQL allows developers to work with dynamic table and column names. This flexibility is beneficial when dealing with scenarios where the table or column names are not known or need to be determined at runtime.
- **Database Administration:** Dynamic SQL can be useful for database administrators (DBAs) when performing tasks such as data migration, schema changes, or automated maintenance operations. Dynamic SQL allows DBAs to generate and execute SQL statements on the fly, adapting to the specific requirements of the task at hand.

Relational Algebra

- A procedural language consisting of a set of operations that take one or two relations as input and produce a new relation as their result.
- Six basic operators
  - select: σ
  - project: ∏
  - union: ∪
  - set difference: –
  - Cartesian product: x
  - rename: ρ

Select Operation

- The **select** operation selects tuples that satisfy a given predicate.
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Example: select those tuples of the *instructor* relation where the instructor is in the "Physics" department.
  - Query
- $\sigma_{dept_name="Physics"}(instructor)$
- Result

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Figure 2.4 Unsorted display of the *instructor* relation.

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Figure 2.10 Result of $\sigma_{dept_name="Physics"}(instructor)$.

Select Operation (Cont.)

- We can use comparison operators like $=, \neq, >, \geq, <, \leq$ in the selection predicate.
- We can combine several predicates into a larger predicate by using the connectives:

  $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)
- Example: Find the instructors in Physics with a salary greater $90,000, we write:

$$\sigma_{dept_name="Physics" \wedge salary > 90,000}(instructor)$$

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Figure 2.4 Unsorted display of the *instructor* relation.

ID	name	dept_name	salary
22222	Einstein	Physics	95000

Project Operation

- A unary operation that returns its argument relation, with certain attributes left out.
- It creates the subset of relation based on the conditions specified. Here, it selects only selected columns/attributes from the relation- vertical subset of relation
- Notation:

$$\prod_{A1,A2,A3\ \ldots.Ak} (r)$$

- where A_1, A_2, …, A_k are attribute names and r is a relation name.
- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets

Project Operation(Cont.)

- Example: eliminate the *dept_name* attribute of *instructor*
- Query*:*

$$\Pi_{ID,\ name,\ salary}\ (instructor)$$

- Result:

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Figure 2.11 Result of $\Pi_{ID,\ name,\ salary}(instructor)$.

Composition of Relational Operations

- The result of a relational-algebra operation is relation and therefore of relational-algebra operations can be composed together into a **relational-algebra expression**.
- Consider the query -- Find the names of all instructors in the Physics department.
- $\prod_{name}(\sigma_{dept_name = \text{“Physics”}}(instructor))$
- Instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation.

name
Einstein
Gold

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Figure 2.4 Unsorted display of the *instructor* relation.

Cartesian Product Operation

- The Cartesian-product operation (denoted by X) allows us to combine information from any two relations.
- R X S returns a relation instance whose schema contains all the fields of R followed by all the fields of S.
- The result of R X S contains tuples <r ,s> (the concatenation of tuples r and s) for each pair of tuples r ϵ R, s ϵ S.
- If relation R has m tuples and relation S has n tuples, then the resultant relation will have m x n tuples.
- For example, if we perform Cartesian product on EMPLOYEE (5 tuples) and DEPT relations (3 tuples), then we will have new tuple with 15 tuples.

Cartesian Product Operation(Cont.)

- Example: the Cartesian product of the relations *instructor* and t*eaches* is written as: *instructor* X *teaches*
- We construct a tuple of the result out of each possible pair of tuples: one from the *instructor* relation and one from the *teaches* relation
- Since the instructor *ID* appears in both relations, we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.
  - *instructor.ID*
  - *teaches.ID*

Cartesian Product illustration

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Instructor relation

X

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

The *teaches* relation.

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Figure 2.12 Result of the Cartesian product *instructor* × *teaches*.

Join Operation

- The Cartesian-Product

  instructor X *teaches*

associates every tuple of instructor with every tuple of teaches.

- Most of the resulting rows have information about instructors who did NOT teach a particular course.
- To get only those tuples of "*instructor* X *teaches* " that pertain to instructors and the courses that they taught, we write:

$$\sigma_{instructor.id = teaches.id} (instructor \times teaches))$$

We get only those tuples of "*instructor* X *teaches*" that pertain to instructors and the courses that they taught.

- The result of this expression, shown in the next slide

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Figure 2.13 Result of $\sigma_{instructor.ID = teaches.ID}(instructor \times teaches)$.

Union Operation

- The union operation allows us to combine two relations
- Notation: $r \cup s$
- For $r \cup s$ to be valid.

1. *r, s* must have the *same* **arity** (same number of attributes)

2. The attribute domains must be **compatible** (example: 2nd column of *r* deals with the same type of values as does the 2nd column of *s*)

- Example: to find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both

$$\prod_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) \cup \prod_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

The *section* relation.

Union Operation

- Result of:

$$\prod_{course_id} (\sigma_{semester=\text{“Fall”} \wedge year=2017} (section)) \cup \prod_{course_id} (\sigma_{semester=\text{“Spring”} \wedge year=2018} (section))$$

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Set Intersection Operation

- The set intersection operation allows us to find tuples that are in both the input relations.
- Notation: $r \cap s$
- Assume:
  - r, s have the *same arity*
  - attributes of r and s are compatible
- Example: Find the set of all courses taught in both the Fall 2017 and the Spring 2018 semesters.

$$\prod_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) \cap \prod_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

- Result

course_id
CS-101

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

The *section* relation.

Set Difference Operation

- The set-difference operation allows us to find tuples that are in one relation but are not in another.
- Notation $r - s$
- Set differences must be taken between **compatible** relations.
  - r and s must have the same arity
  - attribute domains of r and s must be compatible

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

The *section* relation.

Set Difference Operation (Cont.)

- Example: to find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

$$\prod_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) - \prod_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

- Result

course_id
CS-347
PHY-101

Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.

- The assignment operation is denoted by ← and works like assignment in a programming language.

- Example: Find all instructor in the "Physics" and Music department.

  - $Physics \leftarrow \sigma_{dept_name="Physics"}(instructor)$
  - $Music \leftarrow \sigma_{dept_name="Music"}(instructor)$
  - $Physics \cup Music$

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.

Rename Operation

- The results of relational-algebra expressions do not have a name that we can use to refer to them. The rename operator, ρ , is provided for that purpose

- The expression:

  $\rho_x(E)$

  returns the result of expression *E* under the name *x.*

- Another form of the rename operation:

  $\rho_{x(A1,A2,..An)}(E)$

Relational Calculus

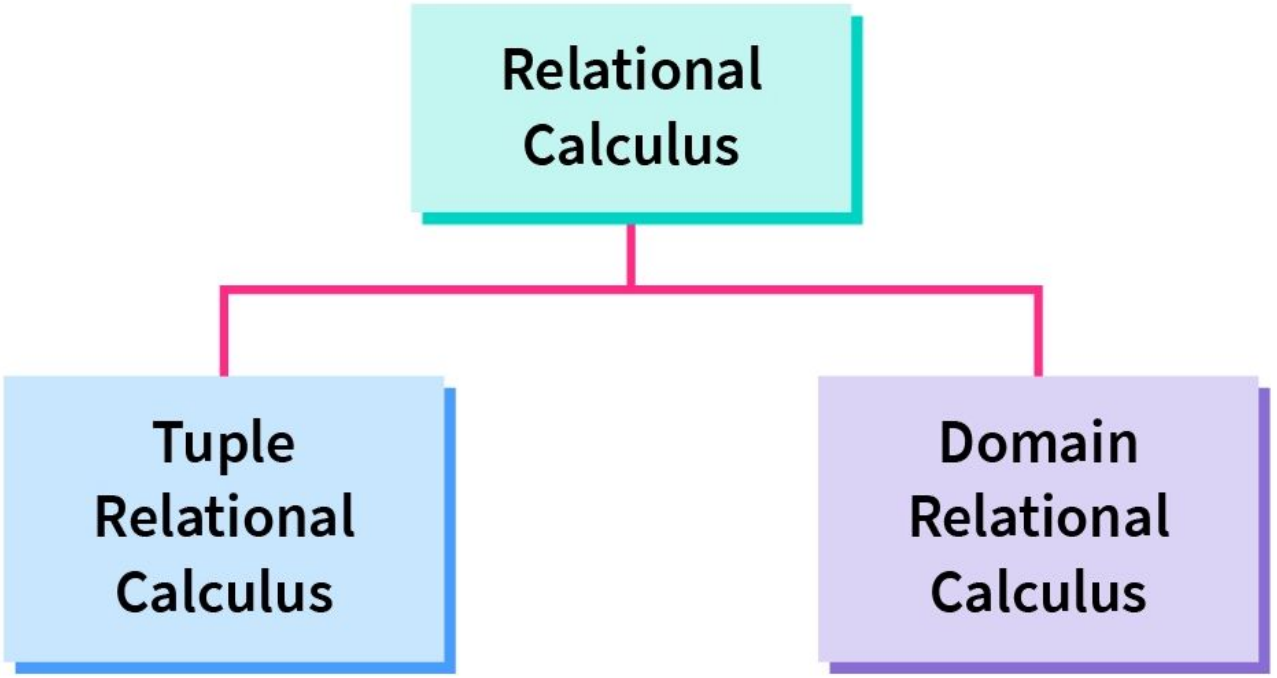
- Relational Calculus is a nonprocedural query language.

- It is based on Predicate calculus, a name derived from branch of symbolic language. A predicate is a truth-valued function with arguments. On substituting values for the arguments, the function result in an expression called a proposition. It can be either true or false. It is a tailored version of a subset of the Predicate Calculus to communicate with the relational database.
- A calculus expression specifies “what to retrieve” rather than “how to retrieve”.
- Most commercial relational languages are based on aspects of relational calculus including SQL-QBE and QUEL.

Difference between Relational Algebra and Relational Calculus

S.No	RELATIONAL ALGEBRA	RELATIONAL CALCULUS
1.	Procedural Language	Non Procedural Language
2	This means “how to obtain the result”	This means “what to obtain” as result.
3.	The order is specified in which operations have to be performed.	Order is not specified.
4.	Independent on the domain	Depends on the domain
5.	The code is nearer to programming language.	The code is not nearer to programming language.

Relational Calculus



Tuple Relational Calculus(TRC)

- It is a non-procedural query language which is based on finding a number of tuple variables also known as range variable for which predicate holds true.
- It describes the desired information without giving a specific procedure for obtaining that information.
- The tuple relational calculus is specified to select the tuples in a relation.
- In TRC, filtering variable uses the tuples of a relation. The result of the relation can have one or more tuples.

Notation

- A Query in the tuple relational calculus is expressed as following notation

{T | P (T)} or {T | Condition (T)}

where

- **T** is the resulting tuples
- **P(T)** is the condition used to fetch T.

Example- TRC

{ T.name | Author(T) AND T.article = 'databas e' }

- **Output:**

This query selects the tuples from the AUTHOR relation. It returns a tuple with 'name' from Author who has written an article on 'database'.

TRC(Cont.)

- TRC (tuple relation calculus) can be quantified. In TRC, we can use Existential ($\exists$) and Universal Quantifiers ($\forall$).

- **For example:**

{ R| $\exists$ T $\in$ Authors(T.article='database' AND R.name=T.name)}

- **Output:** This query will yield the same result as the previous one.

TRC- Example

- **Sailor (sid, sname, rating, age)**
  - **Query 1: Find all the sailors with a rating above 4**

$$\{s \mid SAILOR(s) \text{ AND } s.rating > 4\}$$

  - Here, the condition SAILOR(s) specifies that the range relation of tuple variable **s** is SAILOR.
  - Each SAILOR tuple **s** that satisfies the condition s.rating>4 will be retrieved.
  - This query will retrieve all attribute value for each selected SAILOR tuple **s**. To retrive only some of the attributes, we need to write the specific attribute name in the format "**tuple.attribute**"

TRC- Example(Cont.)

- **Query 2: Retrieve the details of the employees whose salary is greater than 50000.**
  - {e | EMPLOYEE(e) AND e.salary > 50000}
  - Here, if we need only some attributes like Emp_ID and Fname then we can write:
    - {e.Emp_ID, e.Fname | EMPLOYEE(e) AND e.salary > 50000}

- **Query 3: Retrieve the birth date and address of the employee whose name is John B. Smith.**
  - {e.dob, e.address | EMPLOYEE(e) AND e.Fname='John' AND e.Minit='B' AND e.Lname='Smith'}

Expression and Formula in TRC

- Let **Rel** → be a relation name;
  R, S → be the tuple variables;
  a → an attribute of R, **b** → an attribute of S;
  op → operator in the set {<, ≤, >, ≥, =, != }.

- **Rule 1:** An Atomic formula is one of the following:
  - R ∈ Rel
  - R.a op S.b
  - R.a op Constant or Constant op R.a

- **Rule 2:** A formula or boolean condition is made up of one or more atoms connected via the logical operators AND, OR, and NOT i.e. If F1 and F2 are formulas, then so are F1 **AND** F2, F1 **OR** F2, **NOT**(F1) or **NOT**(F2).

The Existential and Universal Quantifiers

- To represent the join and division operation of relational algebra by relational calculus, we need quantifiers such like **Existential quantifier (∃) for join** and **Universal quantifier(∀) for division.**
- A tuple variable **t** is bound if it is quantified i.e. the tuple **t** appears in an **(∃t) or (∀t)** clause; otherwise it is free.
- **Rule 3:** If F is a formula, then so is **(∃t)(F)**, where **t** is a tuple variable. The formula **(∃t)(F)** is **TRUE** if **F evaluates to TRUE** for **at least one** tuple assigned to free occurences of t in F; otherise the formula **(∃t)(F)** is FALSE.
- **Rule 4:** If F is a formula then so is **(∀t)(F)**, where **t** is a tuple variable. This formula **(∀t)(F)** is **TRUE** if **F evaluates to TRUE** for **ALL** the tuples assigned to free occurences of t in F; otherise the formula **(∀t)(F)** is FALSE.

An Example

- **Consider the following relational schema**

  Sailor(sid, sname, rating, age)

  Boat(bid, bname, color)

  Reserves(sid, bid, day)

- **Query 1: Find the names & ages of sailors with a rating above 4**

$$\{ t \mid \exists s \in \text{Sailor}(t.\text{sname} = s.\text{sname} \wedge t.\text{age} = s.\text{age}) \wedge s.\text{rating} > 4 \}$$

How to read: The set of all tuples **t** such that **there exists** a tuple **s** in relation Sailor for which rating is greater than 4.

Sample Queries in TRC

- **Query 2: Find the sailor name, boat id & reservation date for each reservation.**
  - $\{ t \mid \exists s \in Sailor(t.sname = s.sname) \wedge \exists r \in Reserves (t.bid = r.bid \wedge t.day = r.day) \wedge r.sid = s.sid\}$

- **Query 3: Find the names of sailors who have reserved boat 111.**
  - $\{t \mid \exists s \in Sailor(t.sname = s.sname) \wedge \exists r \in Reserves(r.sid = s.sid \wedge r.bid=111)\}$

- **Query 4: Find the names of sailors who have reserved a green boat**
  - $\{t \mid \exists s \in Sailor(t.sname = s.sname) \wedge \exists r \in Reserves(r.sid = s.sid) \wedge \exists b \in Boat(b.bid = r.bid \wedge b.color = 'green'))\}$

contd..

- **Query 5: Find the names of sailors who have reserved at least 2 boats.**
  - $\{t \mid \exists s \in Sailor(t.sname = s.sname) \wedge \exists r1 \in Reserves \wedge \exists r2 \in Reserves \wedge r1.sid = r2.sid \wedge r1.bid \mathrel{!}= r2.bid)\}$

- **Query 6: Find the names of sailors who have reserved all boats.**
  - $\{t \mid \exists s \in Sailor(t.sname = s.sname) \wedge \forall b \in Boat(\exists r \in Reserves(r.sid = s.sid \wedge r.bid = b.bid))\}$

- **Query 7: Find sailors who have reserved all green boats.**
  - $\{t \mid \exists s \in Sailor \wedge \forall b \in Boat(b.color='green' \wedge \exists r \in Reserves(r.sid = s.sid \wedge r.bid = b.bid))\}$

Domain Relational Calculus

- In domain relational calculus, filtering variable uses the domain of attributes.
- Domain relational calculus uses the same operators as tuple calculus.
- It uses logical connectives $\wedge$ (and), $\vee$ (or) and $\neg$ (not). It uses Existential ($\exists$) and Universal Quantifiers ($\forall$) to bind the variable.
- The QBE or Query by example is a query language related to domain relational calculus.

Domain Relational Calculus (DRC)

- In tuple relational calculus, the variables range over the tuples whereas in domain relational calculus, the variables range over the domains.
- The domain variables are the ones which range over the underlying domains instead of over the relations
- The domain relational calculus query has the form:

$$\{<x_1, x_2, \ldots x_n> \mid P(x_1, x_2, \ldots x_n)\}$$

  - where x_i is either a domain variable or a constant and $P(x_1, x_2, x_n)$ is the domain relational calculus formula whose only free variables are the variables among the xi, $1 \leq i \leq n$
- The result of this query is the set of all domains $<x_1, x_2, \ldots x_n>$ for which the formula evaluates to TRUE

DRC Formulas

- *Atomic formula:*
  - $\langle x1, x2, ..., xn \rangle \in Rname$, or X *op* Y, or X *op* constant
  - *op* is one of $<, >, =, \leq, \geq, \neq$
- *Formula:*
  - an atomic formula, or
  - $\neg p, p \wedge q, p \vee q$, where p and q are formulas, or
  - $\exists X\,(p(X))$, where variable X is *free* in p(X), or
  - $\forall X\,(p(X))$, where variable X is *free* in p(X)
- The use of quantifiers $\exists X$ and $\forall X$ is said to *bind* X.
  - A variable that is not bound is free.

Free and Bound Variables

- The use of quantifiers $\exists X$ and $\forall X$ in a formula is said to *bind* X.
  - A variable that is not bound is free.
- Let us revisit the definition of a query:

$$\{\langle x1, x2, ..., xn\rangle \mid p(\langle x1, x2, ..., xn\rangle)\}$$

- There is an important restriction: the variables x1, ..., xn that appear to the left of `|' must be the *only* free variables in the formula p(...).

Queries-DRC

Sailor Database

Sailors(sid, sname, rating, age)

Boats(bid, bname, color)

Reserves(sid, bid, day)

Query: Find all sailors with a rating above 7

$\{<I, N, T, A> / <I, N, T, A> \in \text{Sailors} \wedge T > 7\}$

Query: Find the names of sailors who reserved boat 111

$\{<N> / \exists I, T, A\ (<I, N, T, A> \in \text{Sailors} \wedge \exists <Ir, Br, D> \in \text{Reserves}\ (Ir=I \wedge Br=111))\}$

This query can also be written as:

$\{<N> / \exists I, T, A\ (<I, N, T, A> \in \text{Sailors} \wedge \exists <Ir, Br, D> \in \text{Reserves}\ (Ir=I \wedge Br=111))\}$

or

$\{<N> / \exists I, T, A\ (<I, N, T, A> \in \text{Sailors} \wedge \exists D(<I, 111, D> \in \text{Reserves}))\}$

Queries-DRC

Query: Find the names of sailors who have reserved a green boat

$\{\langle N\rangle / \exists I, T, A\ (\langle I, N, T, A\rangle \in \text{Sailors} \wedge \langle I, Br, D\rangle \in \text{Reserves} \wedge \exists \langle Br, Bn, \text{'green'}\rangle \in \text{Boats})\}$

Query: Find the names of sailors who have reserved at least 2 boats

$\{\langle N\rangle / \exists I, T, A\ (\langle I, N, T, A\rangle \in \text{Sailors} \wedge \exists Br1, Br2, D1, D2(\langle I, Br1, D1\rangle \in \text{Reserves} \wedge \langle I, Br2, D2\rangle \in \text{Reserves} \wedge Br1\ 6{=}\ Br2))\}$

Query: Find the names of sailors who have reserved all boats

$\{\langle N\rangle / \exists I, T, A(\langle I, N, T, A\rangle \in \text{Sailors} \wedge \forall \langle B, Bn, C\rangle \in \text{Boats}\ (\exists \langle Ir, Br, D\rangle \in \text{Reserves}\ (I{=}Ir \wedge Br{=}B)))\}$

Query: Find sailors who have reserved all green boats

$\{\langle I,N,T,A\rangle / \langle I,N,T,A\rangle \in \text{Sailors} \wedge \forall \langle B, Bn, C\rangle \in \text{Boats}\ (C{=}\text{'green'} \Rightarrow\ \exists \langle Ir, Br, D\rangle \in \text{Reserves}\ (I{=}Ir \wedge Br{=}B)))\}$

Entity Relationship Model

- The ER data model was developed to facilitate database design by allowing specification of an **enterprise schema** that represents the overall logical structure of a database.

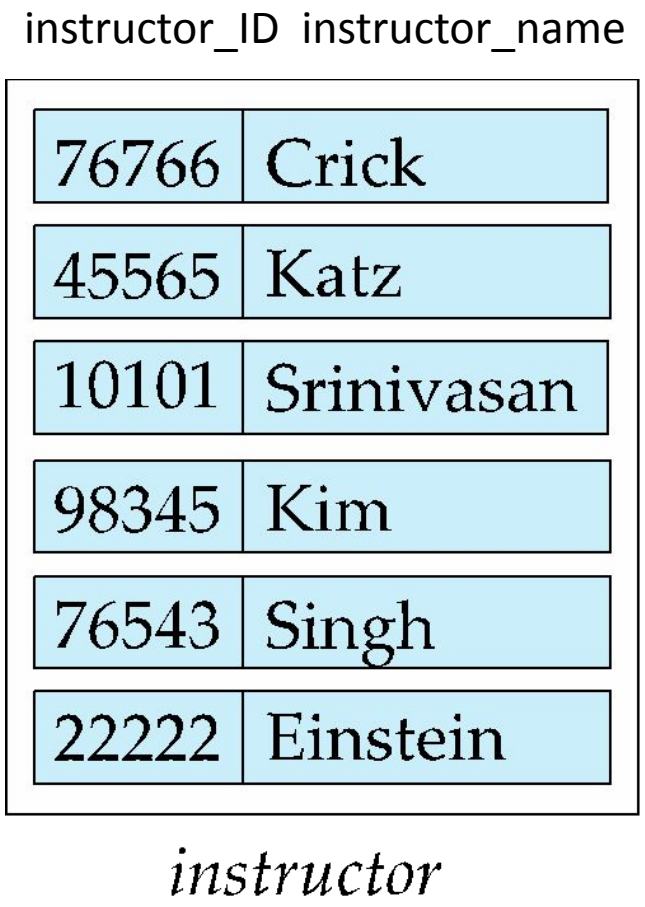
- The ER data model employs three basic concepts:
  - entity sets,
  - relationship sets,
  - attributes.

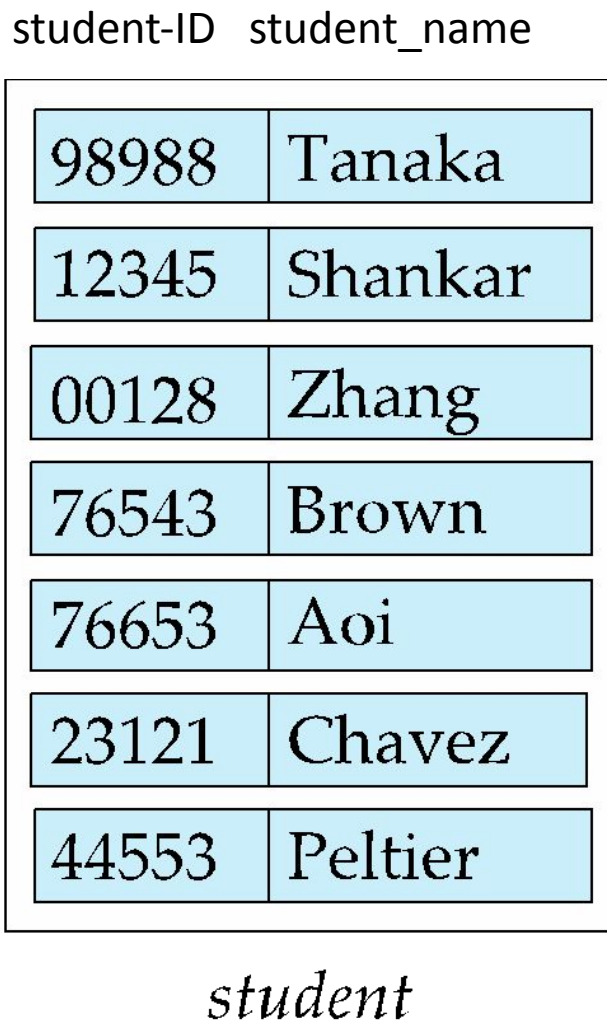
- The ER model also has an associated diagrammatic representation, the **ER diagram**, which can express the overall logical structure of a database graphically.

Entity Sets

- An **entity** is an object that exists and is distinguishable from other objects.
  - Example: specific person, company, event, plant
- An **entity set** is a set of entities of the same type that share the same properties.
  - Example: set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set.
  - Example: *instructor* = (*ID, name, salary*) *course*= (*course_id, title, credits*)
- A subset of the attributes form a **primary key** of the entity set; i.e., uniquely identifying each member of the set.

Entity Sets -- *instructor* and *student*





Representing Entity sets in ER Diagram

- Entity sets can be represented graphically as follows:
  - Rectangles represent entity sets.
  - Attributes listed inside entity rectangle
  - Underline indicates primary key attributes

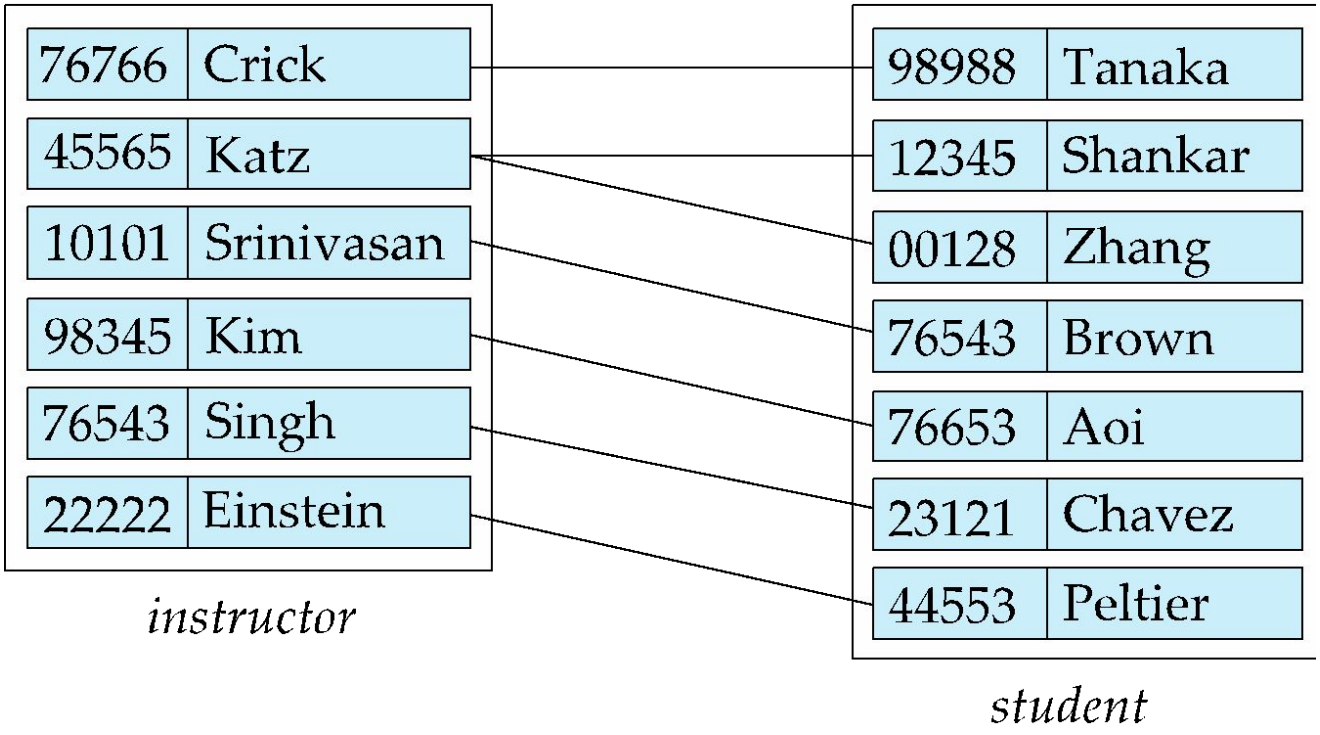
Relationship Sets

- A **relationship** is an association among several entities
- Example:
  44553 (Peltier) *advisor* 22222 (Einstein)
  student entity relationship set *instructor* entity
- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets
- $\{(e_1, e_2, \ldots e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$

  where $(e_1, e_2, \ldots, e_n)$ is a relationship
- Example:
- (44553,22222) $\in$ *advisor*
-

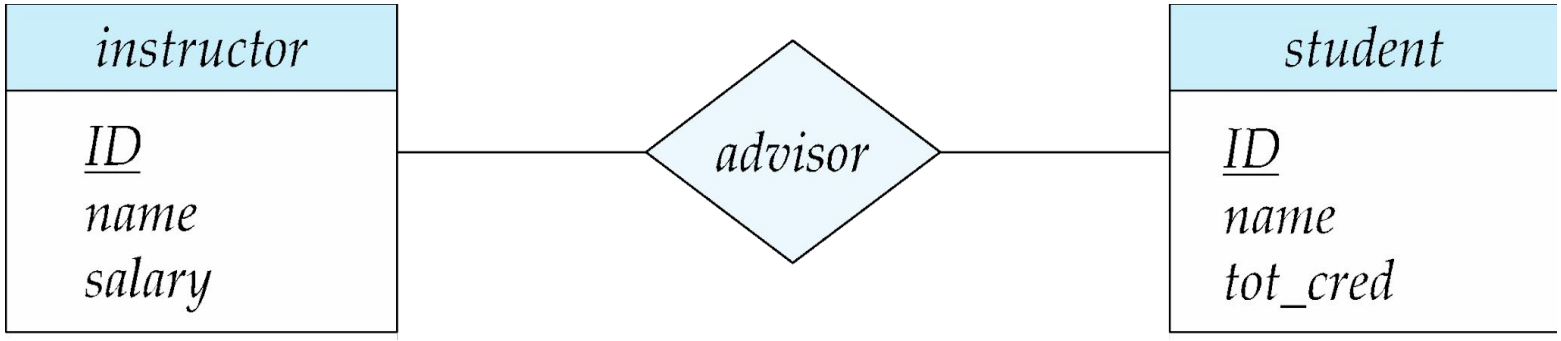
Relationship Sets

- Example: we define the relationship set *advisor* to denote the associations between students and the instructors who act as their advisors.
- Pictorially, we draw a line between related entities.



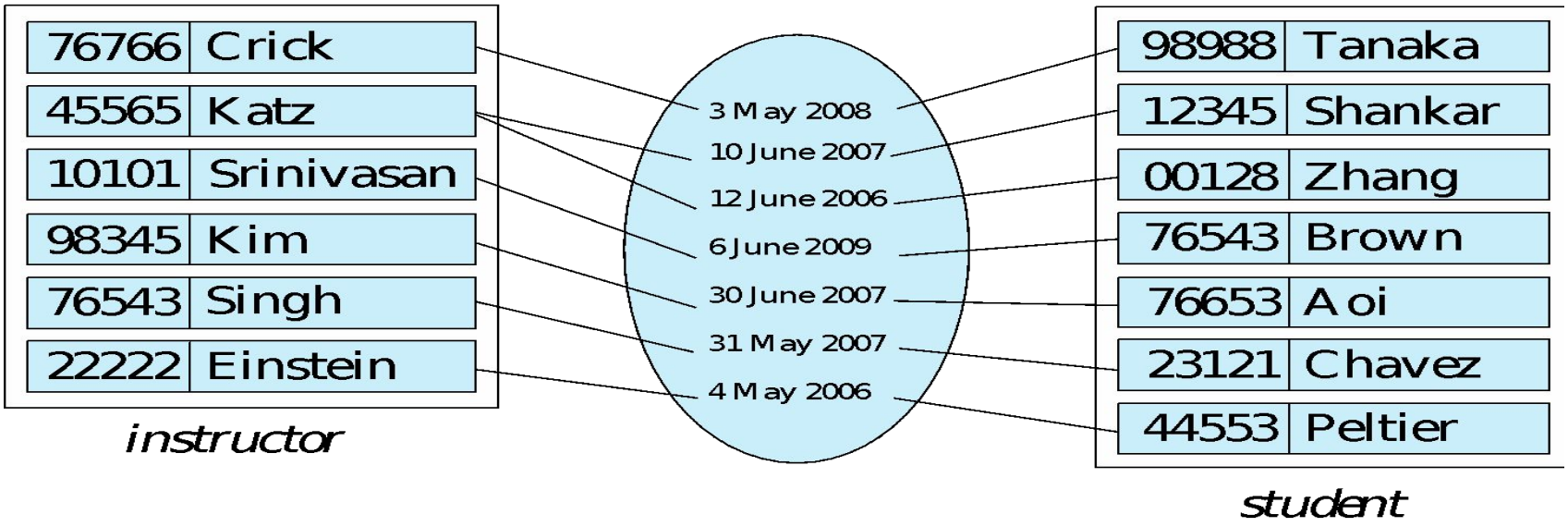
Representing Relationship Sets via ER Diagrams

- Diamonds represent relationship sets.

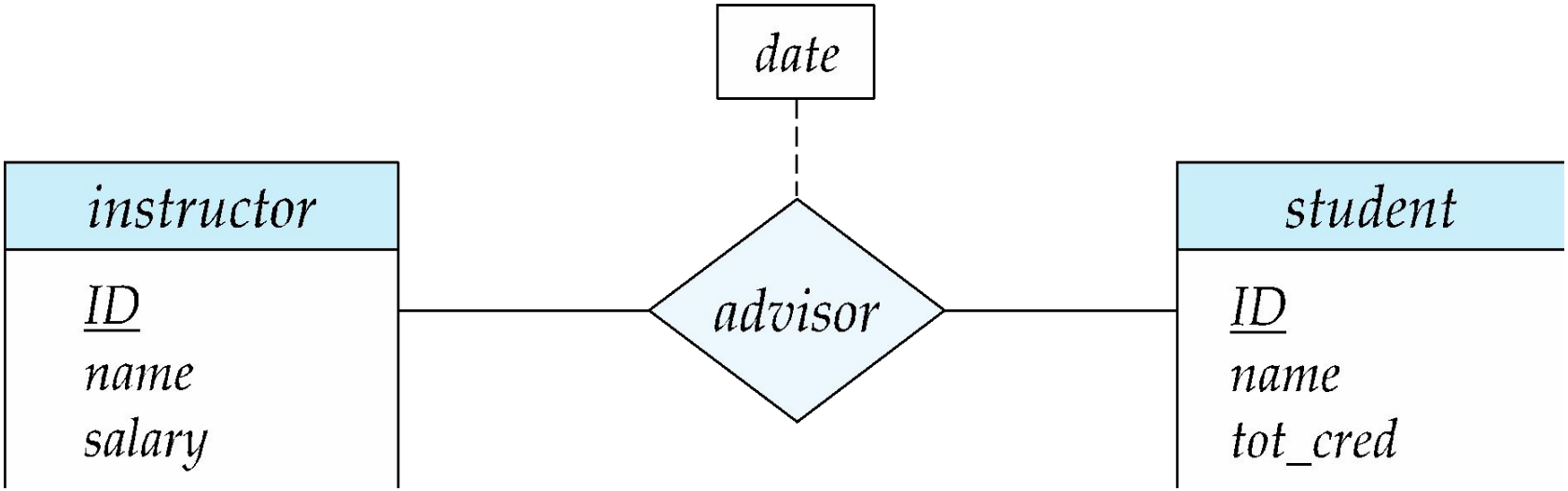


Relationship Sets

- An attribute can also be associated with a relationship set.
- For instance, the *advisor* relationship set between entity sets *instructor* and *student* may have the attribute *date* which tracks when the student started being associated with the advisor



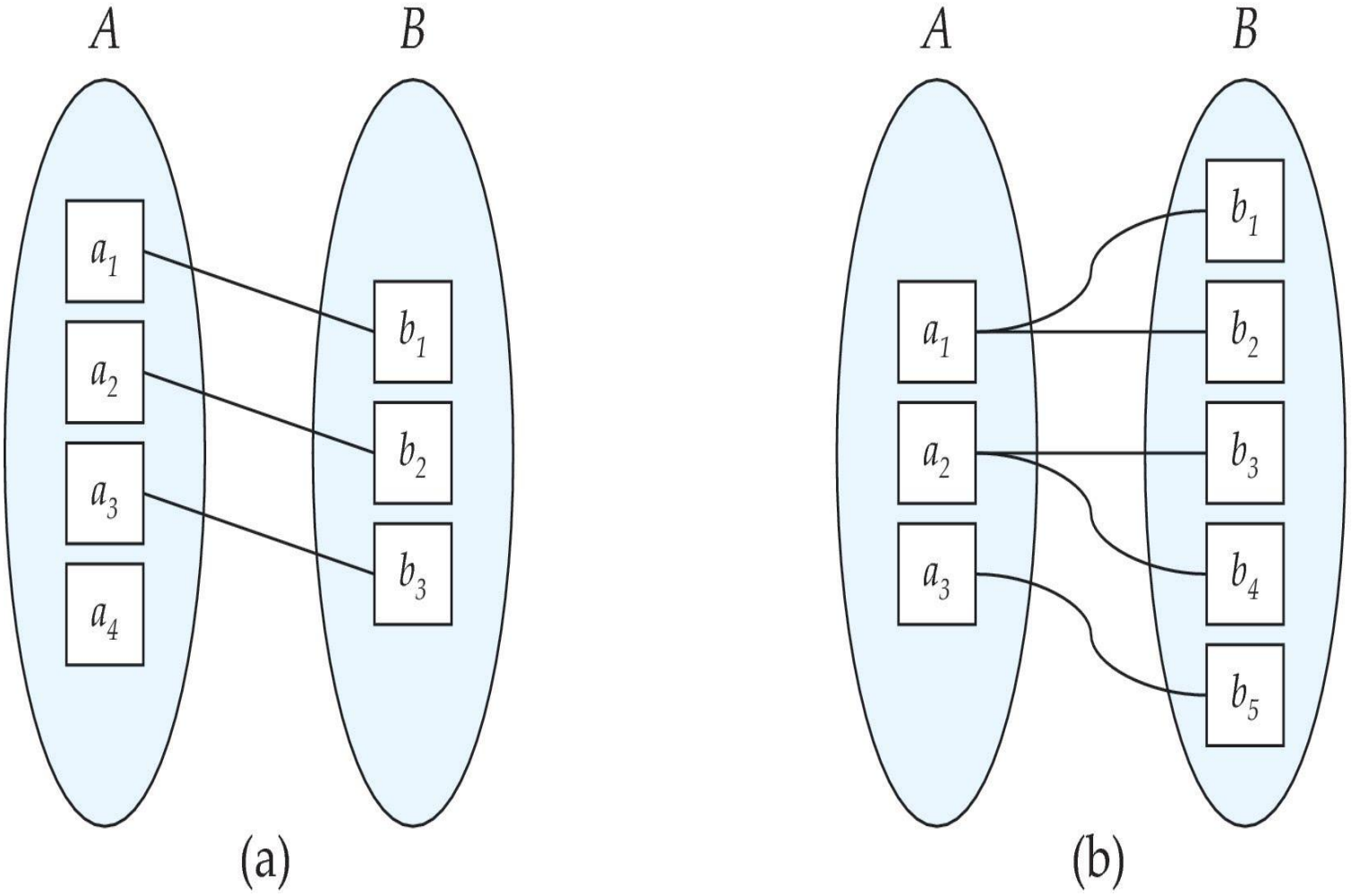
Relationship sets with Attributes



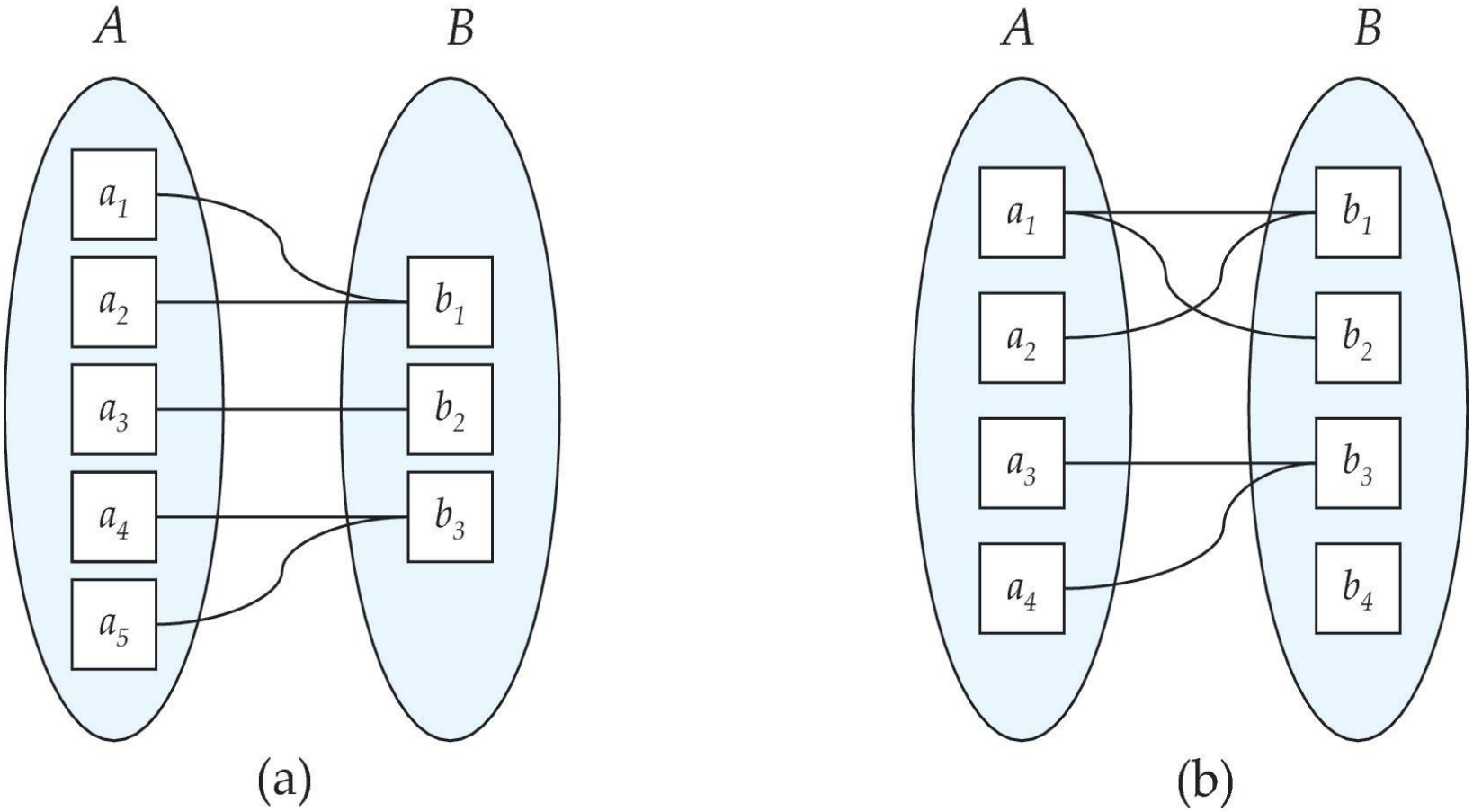
Mapping Cardinality Constraints

- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
  - One to one
  - One to many
  - Many to one
  - Many to many

Mapping Cardinality Constraints



Mapping Cardinality Constraints



One to One Relationship



One-to-Many relationship

- one-to-many relationship between an *instructor* and a *student*
  - an instructor is associated with several (including 0) students via *advisor*
  - a student is associated with at most one instructor via advisor,



Many to Many Relationship



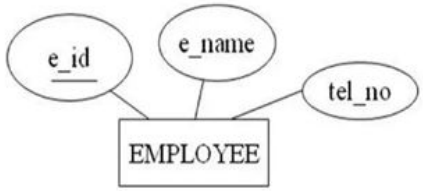
Many-to-One relationship

- In a many-to-one relationship between an *instructor* and a *student,*
  - an instructor is associated with at most one student via *advisor*,
  - and a student is associated with several (including 0) instructors via *advisor*

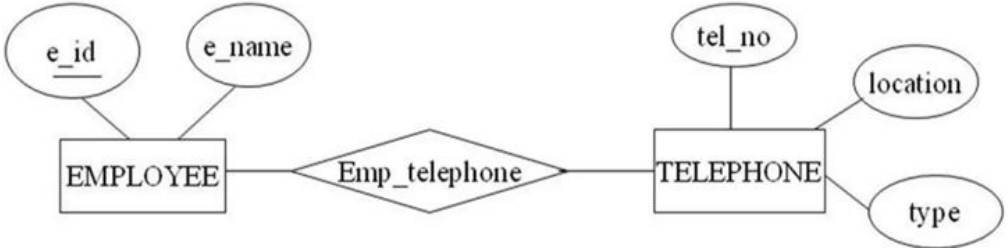
Design Issues of ER Model

ER diagram design issues need to be discussed for better ER- design.

1. **Use of Entity set vs. Attributes:**



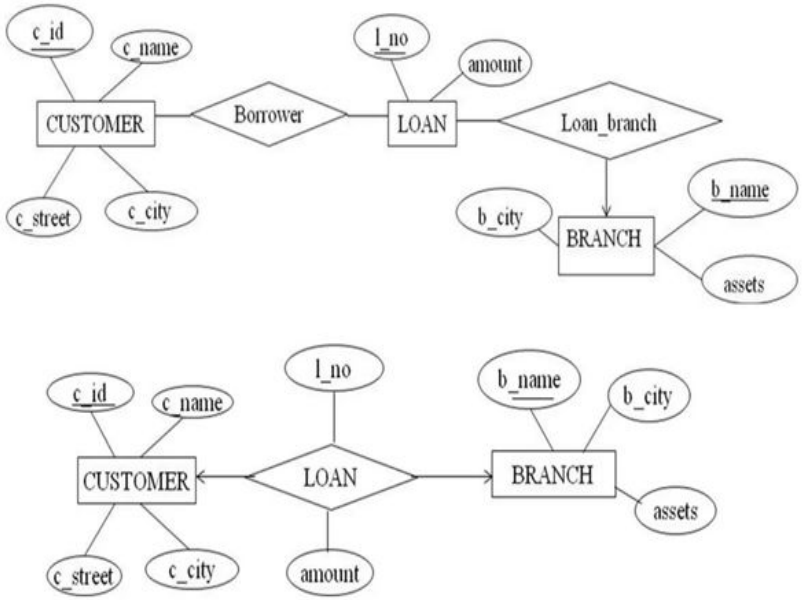




Design Issues of ER Model(Cont.)

2. **Use of Entity sets vs. Relationship sets**

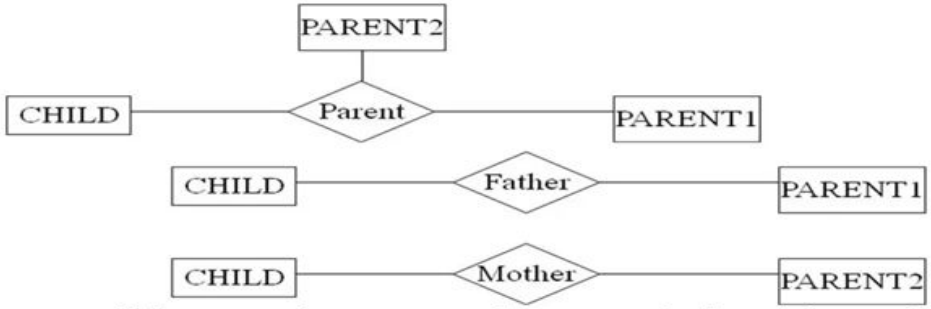
   Sometimes, an entity set can be better expressed in relationship set. Thus, it is not always clear whether an object is best expressed by an entity set or a relationship set.



Design Issues of ER Model(Cont.)

3. **Binary vs. n-ary relationship sets**

Relationships in databases are often binary. Some relationships that appear to be non-binary could actually be better represented by several binary relationships.



- It is always possible to replace a non-binary relationship set by a number of distinct binary relationship sets. For example, consider a ternary relationship R associated with three entity sets A, B and C. We can replace the relationship set R by an entity set E and create three relationship sets as:
  - ✓ RA, relating E and A
  - ✓RB , relating E and B
  - ✓RC , relating E and C
- If the relationship set R had any attributes, these are assigned to entity set E. A special identifying attribute is created for E.

Design Issues of ER Model(Cont.)

4. Placement of Relationship Attributes

The cardinality ratio of a relationship can affect the placement of relationship attributes:

- ✓ **One-to-Many:** Attributes of 1:M relationship set can be repositioned to only the entity set on the many side of the relationship.
- ✓ **One-to-One:** The relationship attribute can be associated with either one of the participating entities.
- ✓ **Many-to-Many:** Here, the relationship attributes can not be represented to the entity sets; rather they will be represented by the entity set to be created for the relationship set.

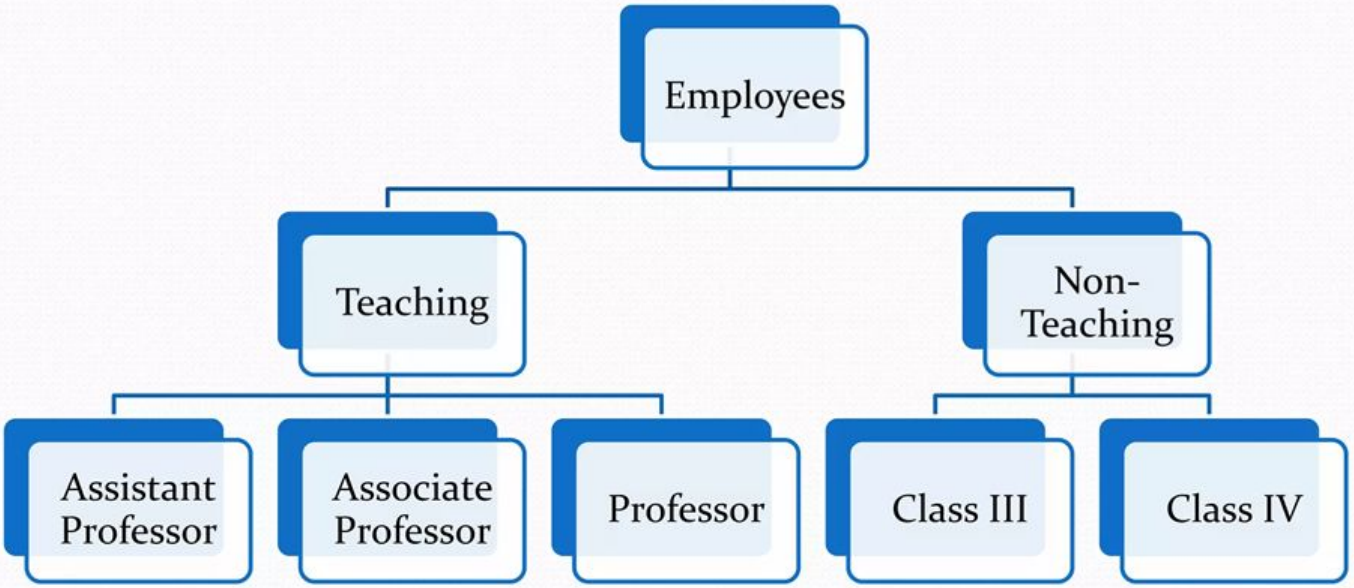
ER Diagram Design Methodologies

- The guidelines that should be followed while designing an ER diagram are discussed below:
  - ✓ Recognize entity sets
  - ✓ Recognize relationship sets and participating entity sets
  - ✓ Recognize attributes of entity sets and attributes of relationship sets
  - ✓ Define binary relationship types and existence dependencies
  - ✓ Define general cardinality, constraints, keys, and discriminators
  - ✓ Design diagram

Extended E-R Features

- Specialization
- Generalization
- higher- and lower-level entity sets
- Attribute inheritance
- Aggregation.

Specialization



Generalisation

- The design process may also proceed in a **bottom-up manner, in which multiple entity** sets are synthesized into a higher-level entity set on the basis of common features.
- Generalization is a simple inversion of specialization.

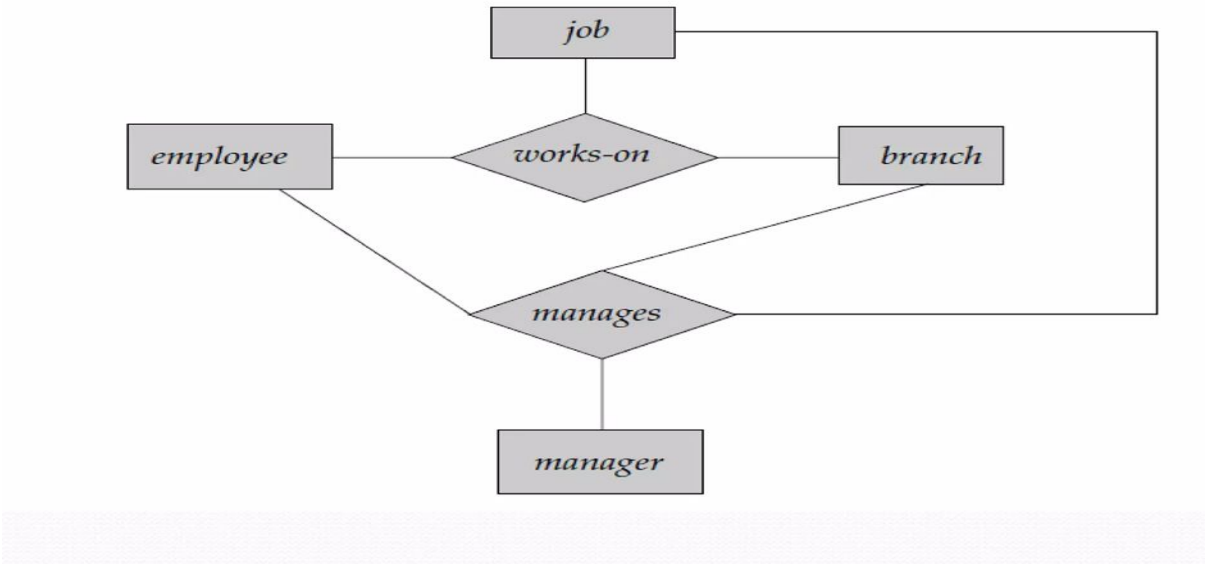
Attribute Inheritance

- A crucial property of the higher- and lower-level entities created by specialization and generalization is **attribute inheritance.**
- The attributes of the higher-level entity sets are said to be inherited by the lower-level entity sets.
- E.g. 'Assistant Prof.' And 'Associate Prof.' Inherit the attributes of 'Teaching staff'.

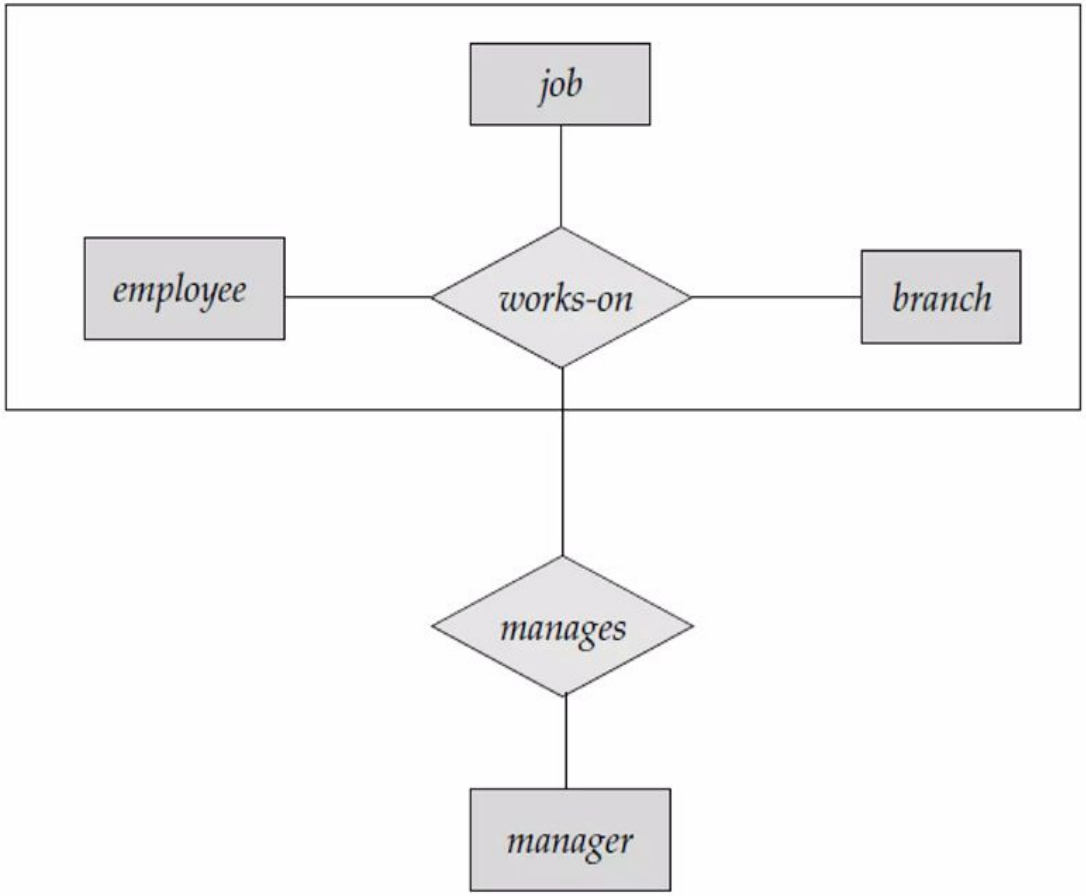
Aggregation

- One limitation of the E-R model is that it cannot express relationships among relationships.
- Aggregation is an abstraction through which relationships are treated as higher level entities.

ER Diagram



E-R with Aggregation

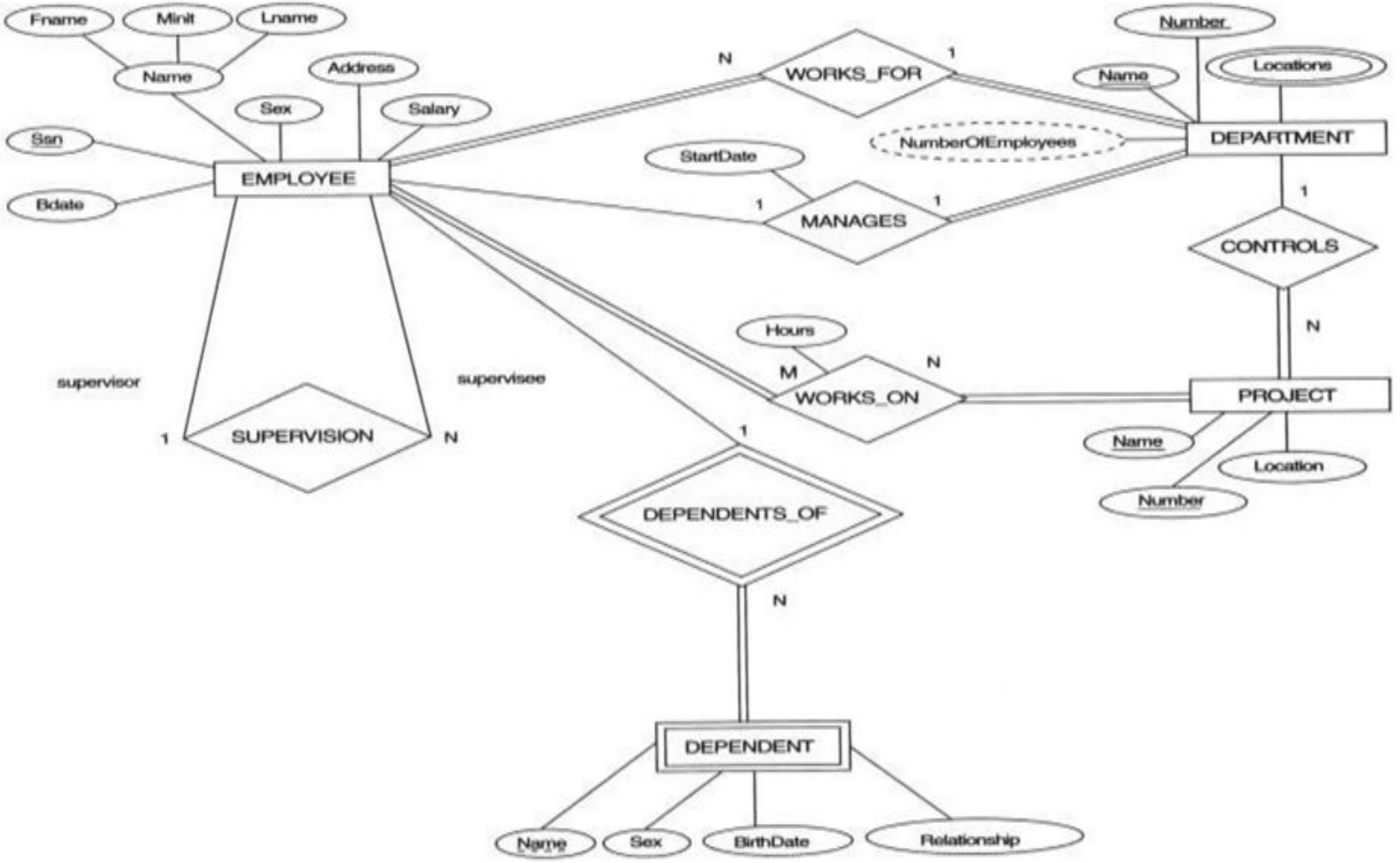


ER to Relational Mapping – Algorithm

7-Step Process:

1. Map **Regular Entity** Types
2. Map **Weak Entity** Types
3. Map **Binary 1:1 Relation** Types
4. Map **Binary 1:N Relationship** Types.
5. Map **Binary M:N Relationship** Types.
6. Map **Multivalued attributes.**
7. Map **N-ary Relationship** Types.

ER Model


Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
NumberOfEmployees
StartDate
1
MANAGES
1
1
CONTROLS
N
Hours
M
N
WORKS_ON
PROJECT
Name
Location
Number
supervisor
supervisee
1
SUPERVISION
N
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

Step 1: Mapping of Regular Entity Types

- For each regular (strong) entity type E in the ER schema, create a relation R that includes all the simple attributes of E.
- Choose one of the key attributes of E as the primary key for R.
- If the chosen key of E is composite, the set of simple attributes that form it will together form the primary key of R.

Mapping of Regular Entity Sets

EMPLOYEE							
Fname	Minit	Lname	Sex	Address	Salary	Ssn	Bdate

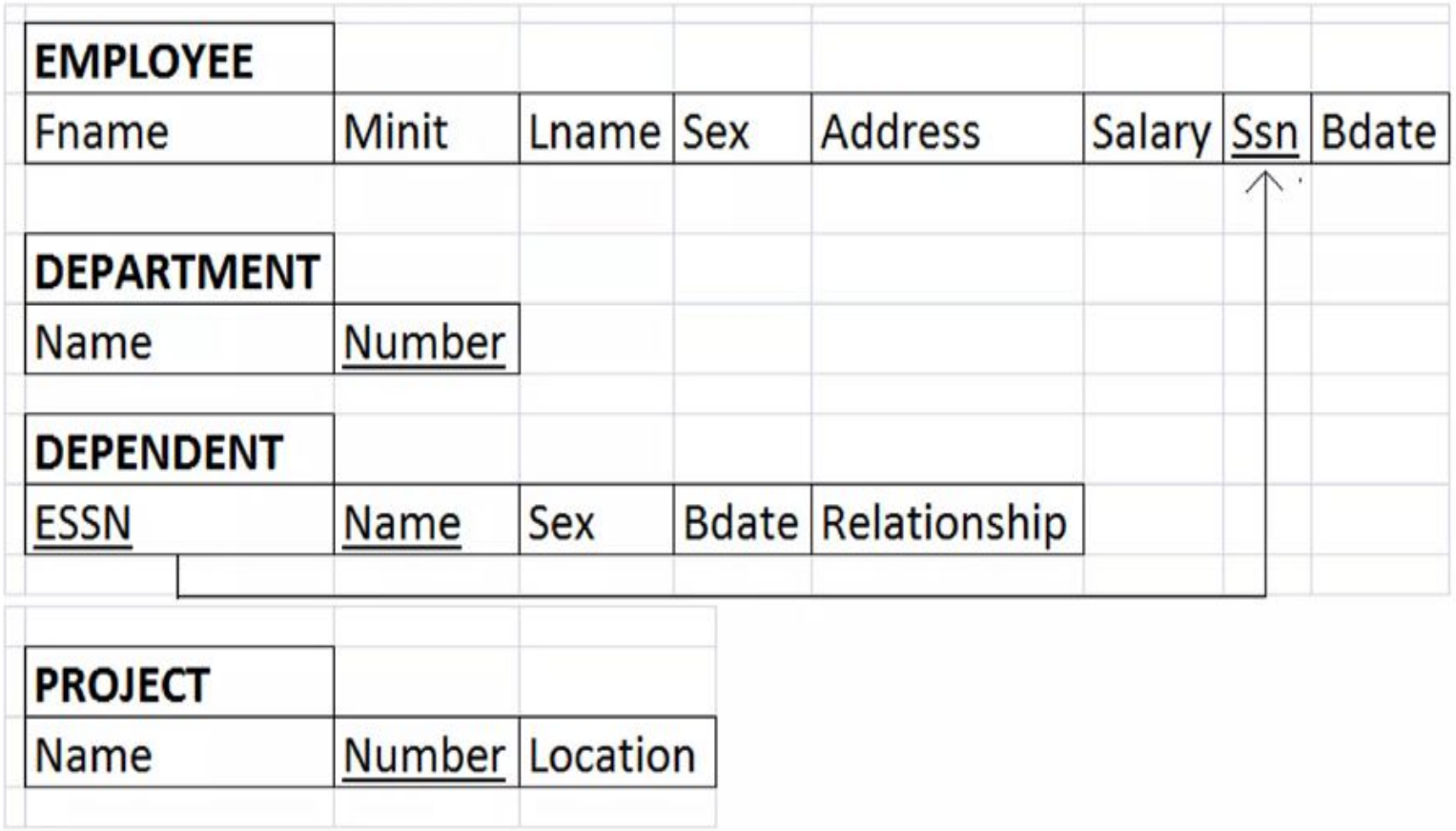
DEPARTMENT	
Name	Number

PROJECT		
Name	Number	Location

Step 2: Mapping of Weak Entity Types

- For each weak entity type W in the ER schema with owner entity type E, create a relation R & include all simple attributes (or simple components of composite attributes) of W as attributes of R.
- Also, include as foreign key attributes of R the primary key attribute(s) of the relation(s) that correspond to the owner entity type(s).
- The primary key of R is the *combination of* the primary key(s) of the owner(s) and the partial key of the weak entity type W, if any.

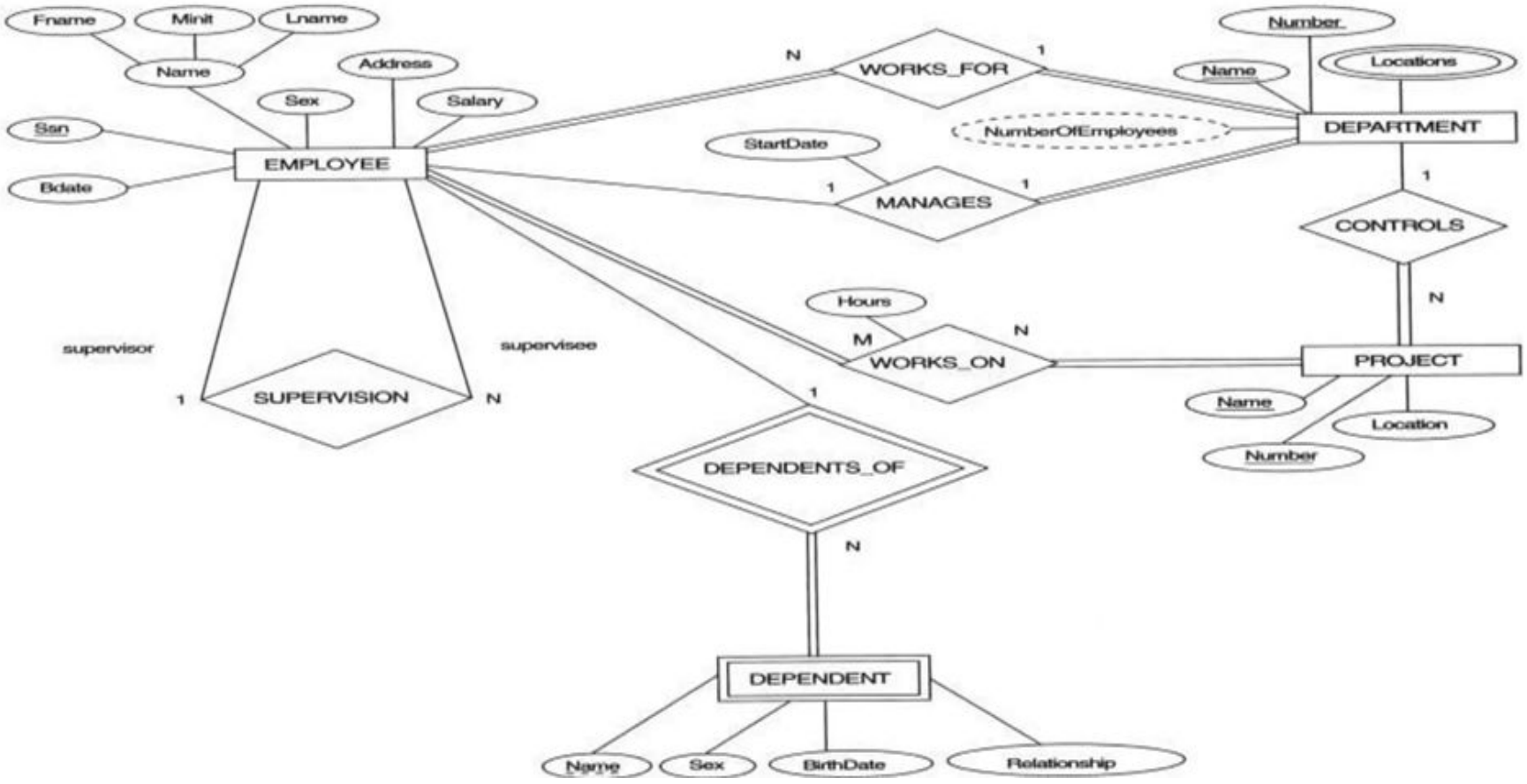
Mapping of Weak Entity Sets



Step 3: Mapping of Binary 1:1 Relation Types

- For each binary 1:1 relationship type R in the ER schema, identify the relations S and T that correspond to the entity types participating in R.
- Choose one of the relations–say S–and include a foreign key in S the primary key of T. It is better to choose an entity type with **total** participation in R in the role of S.

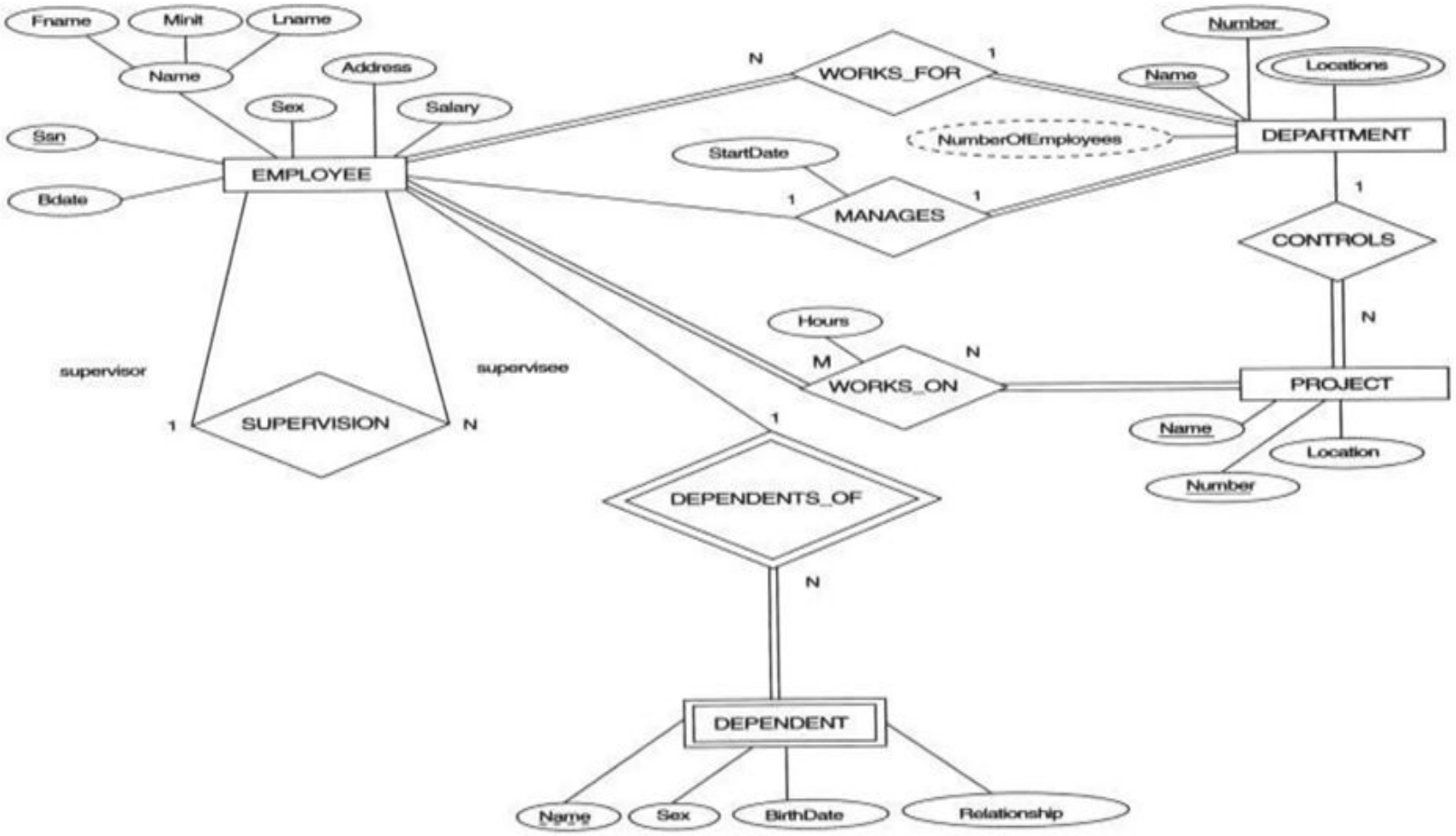
ER Model


Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
NumberOfEmployees
StartDate
1
MANAGES
1
1
CONTROLS
N
Hours
M
N
WORKS_ON
PROJECT
Name
Location
Number
supervisor
supervisee
1
SUPERVISION
N
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

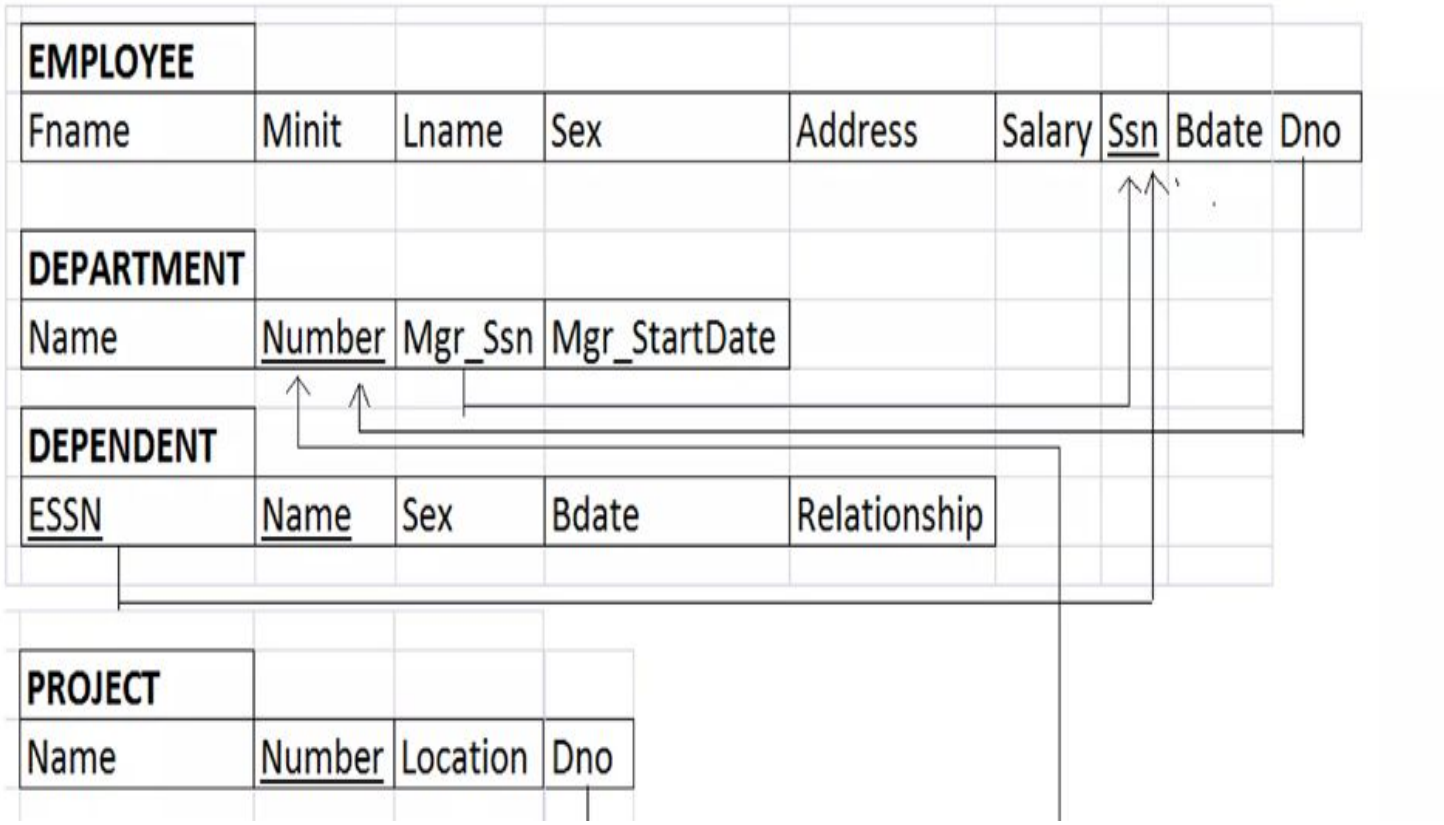
Step 4: Mapping of Binary 1:N Relationship Types

- For each regular binary 1:N relationship type R, identify the relation S that represent the participating entity type at the N-side of the relationship type.
- Include as foreign key in S the primary key of the relation T that represents the other entity type participating in R.
- Include any simple attributes of the 1:N relation type as attributes of S.

ER Model


Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
supervisor
supervisee
1
SUPERVISION
N
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
NumberOfEmployees
StartDate
1
MANAGES
1
1
CONTROLS
N
Hours
M
N
WORKS_ON
PROJECT
1
Name
Location
Number
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

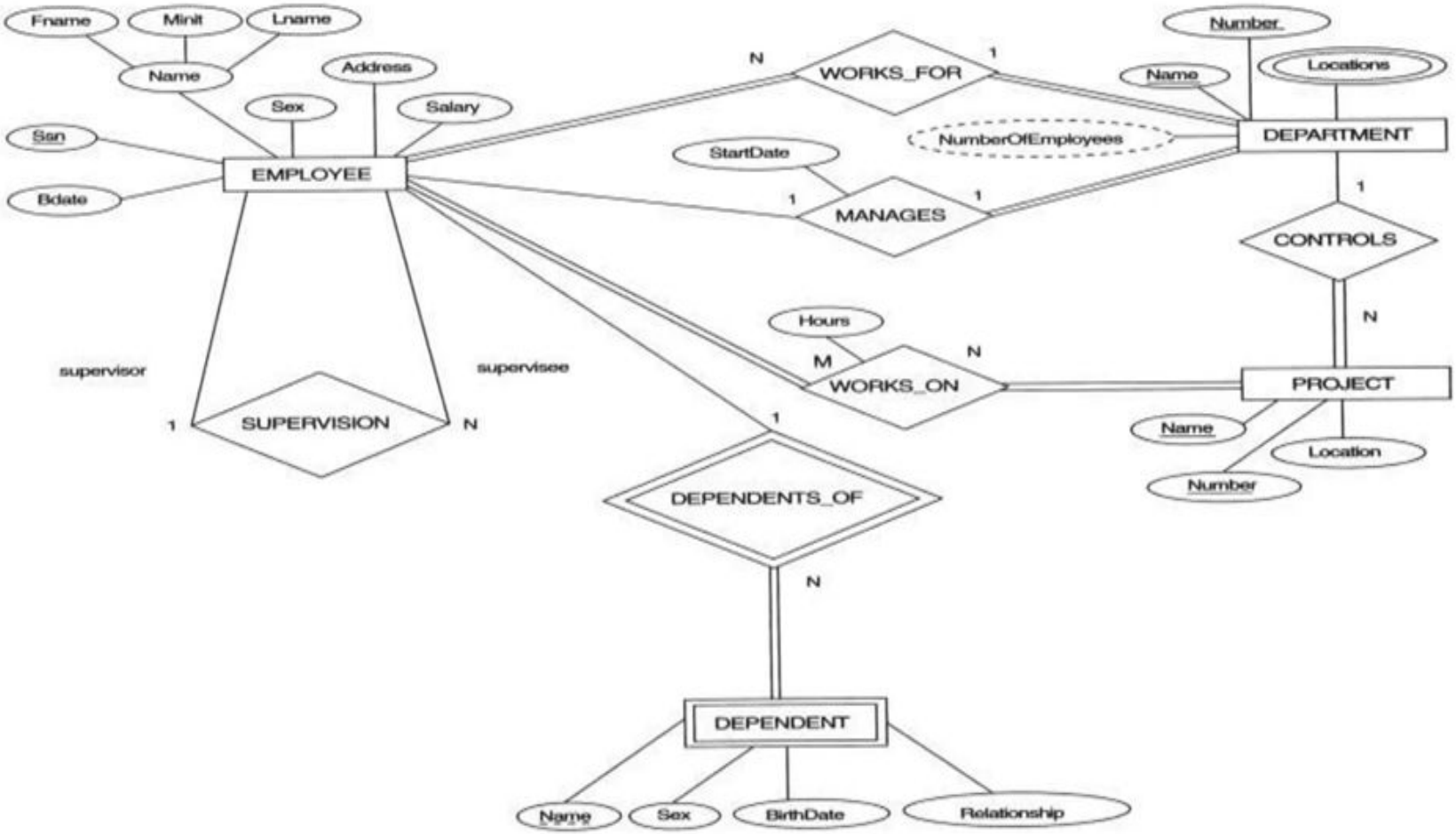
Mapping of Binary 1:N Relationship Types



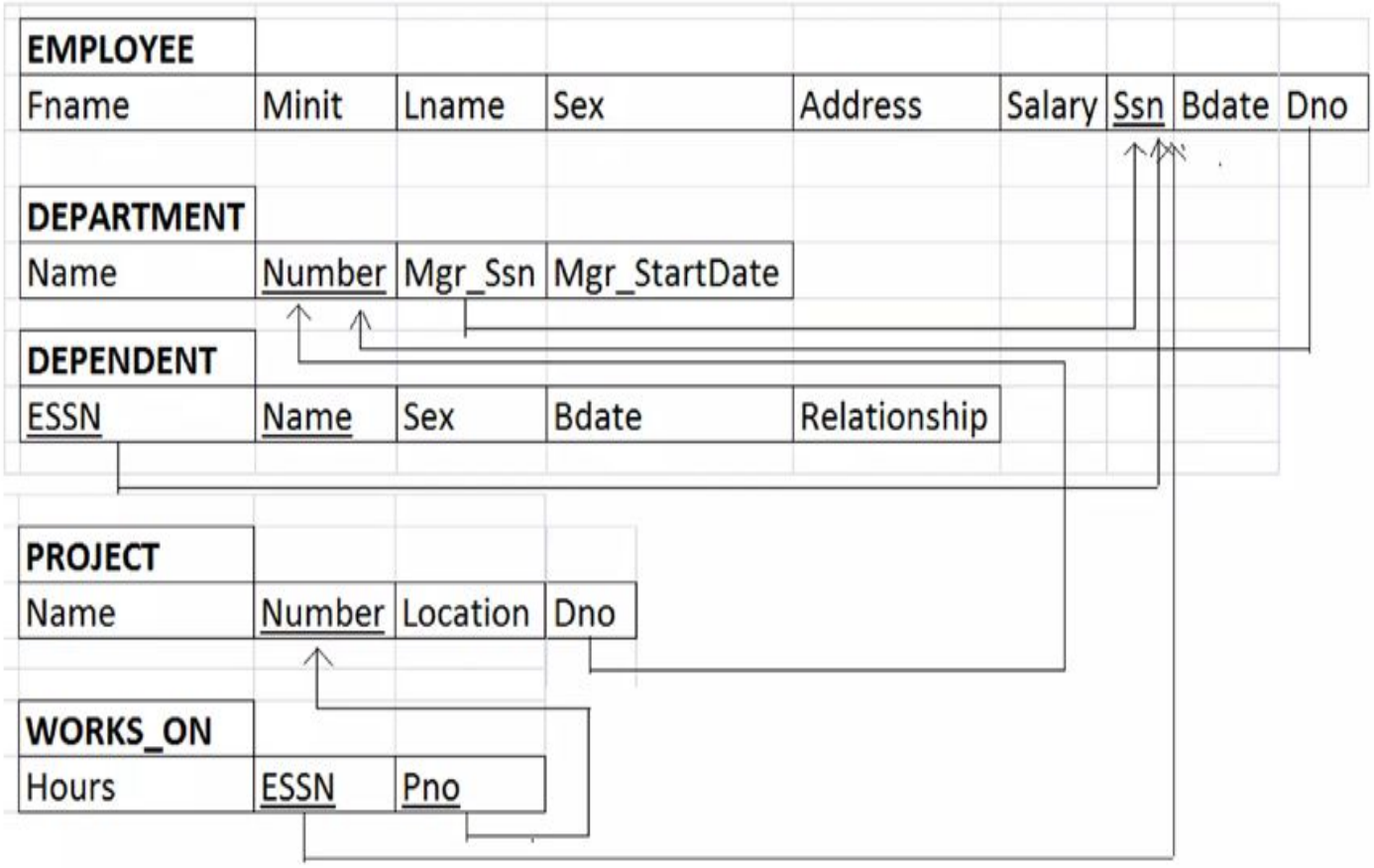
Step 5: Mapping of Binary M:N Relationship Types

- For each regular binary M:N relationship type R, *create new relation* S to represent R.
- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types; *their combination will form the primary key* of S.
- Also include any simple attributes of the M:N relationship type (or simple components of composite attributes) as attributes of S.

ER Model


Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
NumberOfEmployees
StartDate
1
MANAGES
1
1
CONTROLS
N
Hours
M
N
WORKS_ON
PROJECT
Name
Location
Number
supervisor
supervisee
1
SUPERVISION
N
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

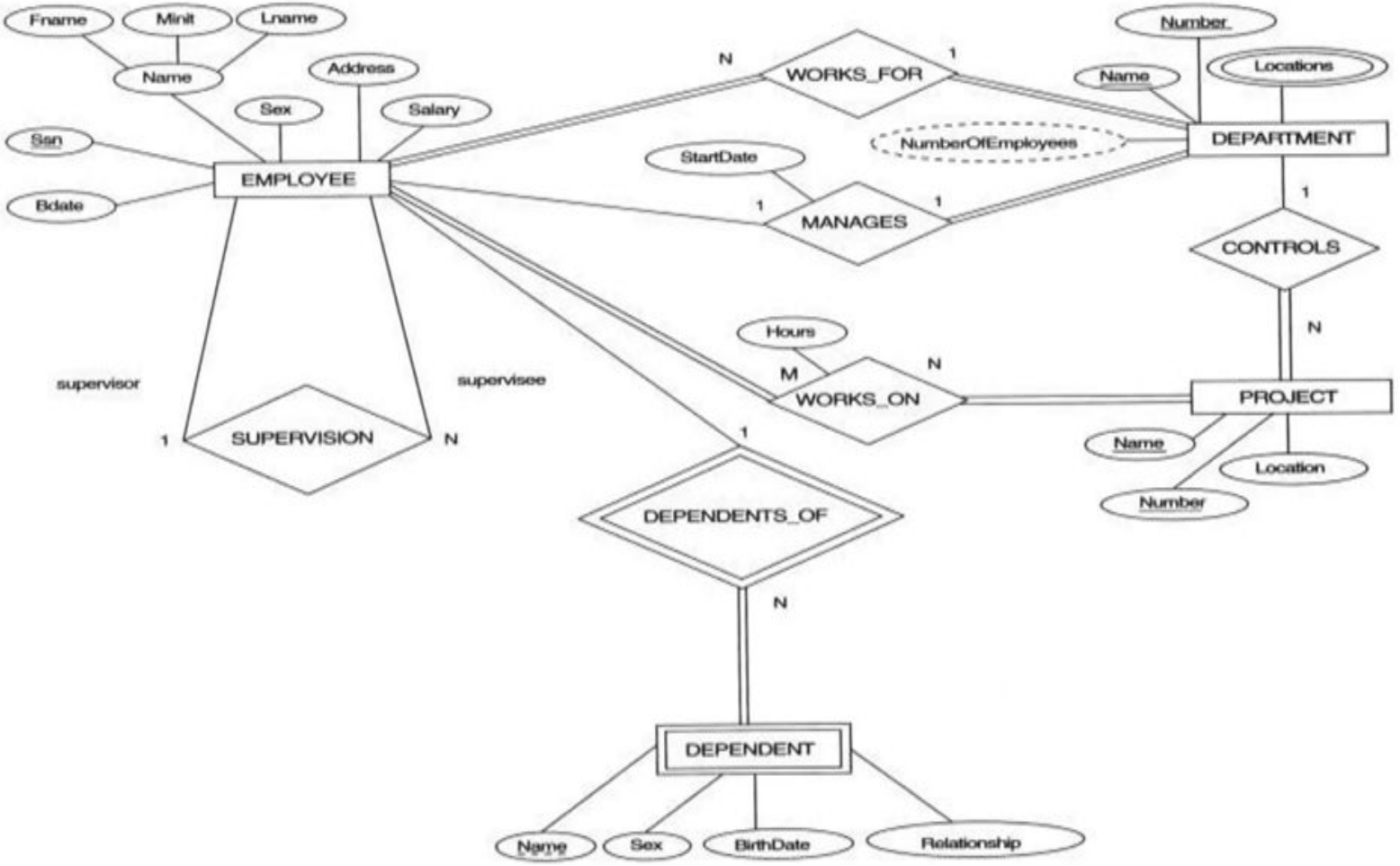
Mapping of Binary M:N Relationship Types



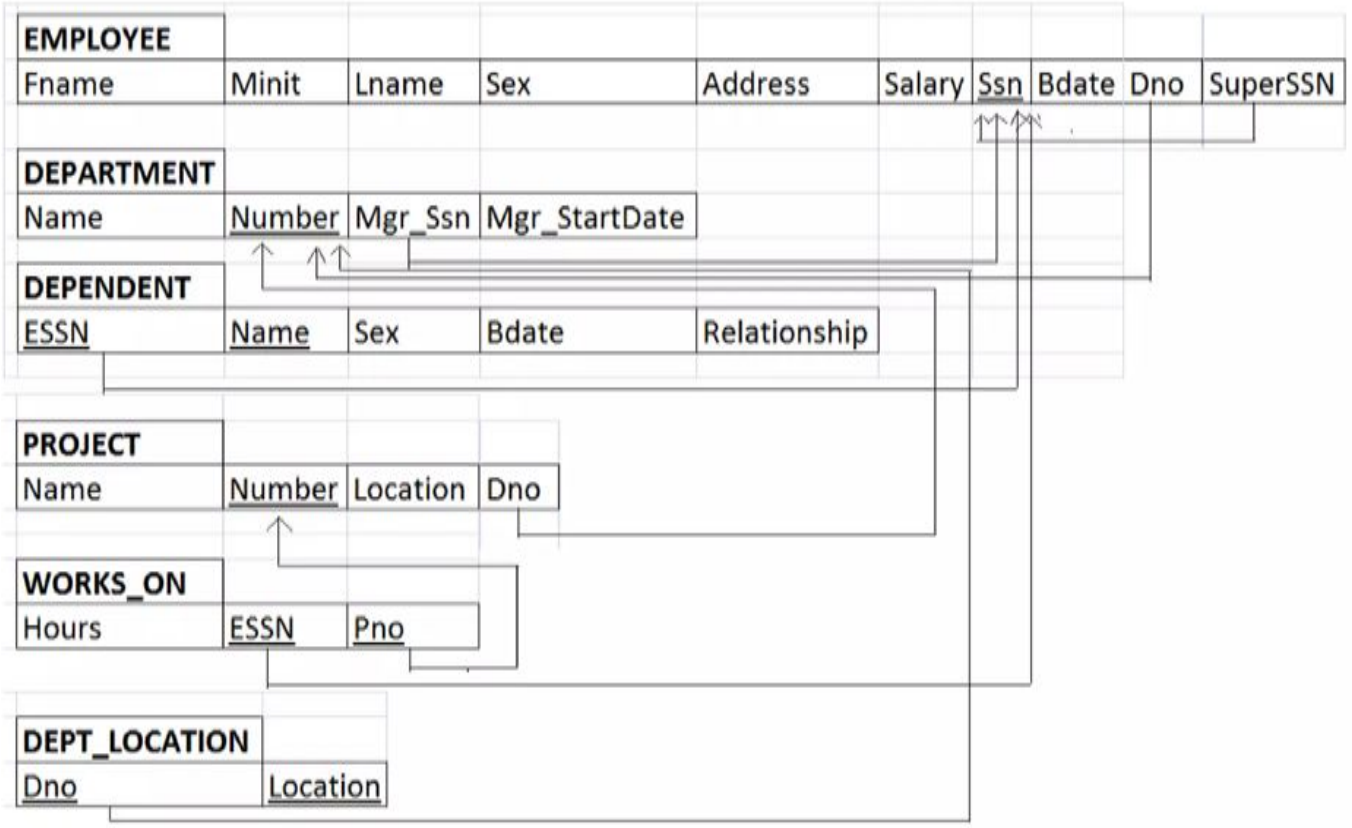
Step 6: Mapping of Multivalued attributes

- For each multivalued attribute A, create a new relation R.
- This relation R will include an attribute corresponding to A, plus the primary key attribute K–as a foreign key in R–of the relation that represents the entity type of relationship type that has A as an attribute.
- The primary key of R is the combination of A and K. If the multivalued attribute is composite, we include its simple components.

ER Model


Fname
Minit
Lname
Name
Address
Sex
Salary
Ssn
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
NumberOfEmployees
StartDate
1
MANAGES
1
1
CONTROLS
N
Hours
M
N
WORKS_ON
PROJECT
Name
Location
Number
supervisor
supervisee
1
SUPERVISION
N
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

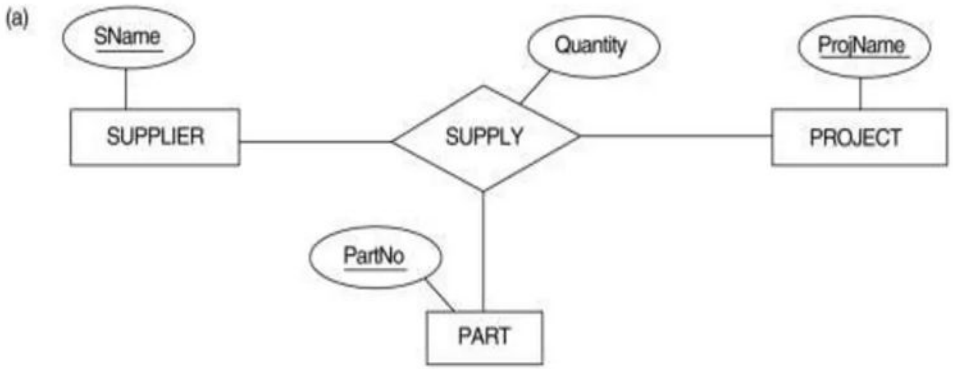
Mapping of Multivalued attributes

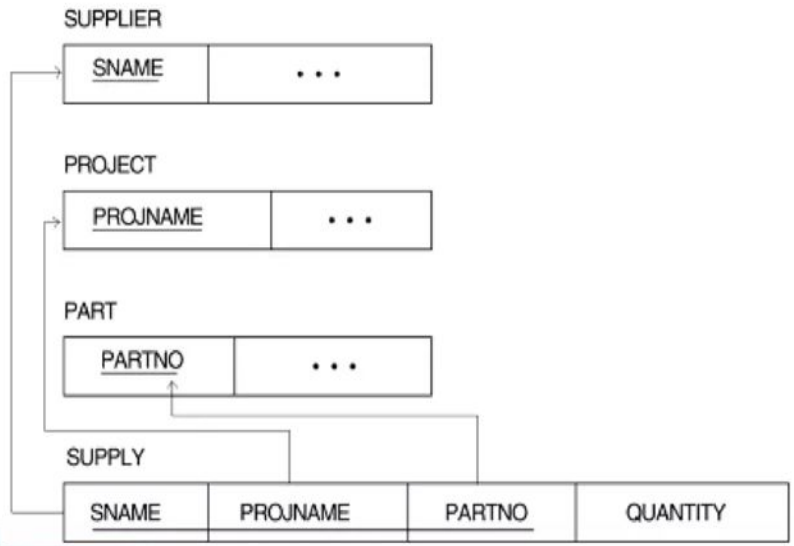


Step 7: Mapping of N-ary Relationship Types

- For each n-ary relationship type R, where n>2, create a new relationship S to represent R.
- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
- Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S.

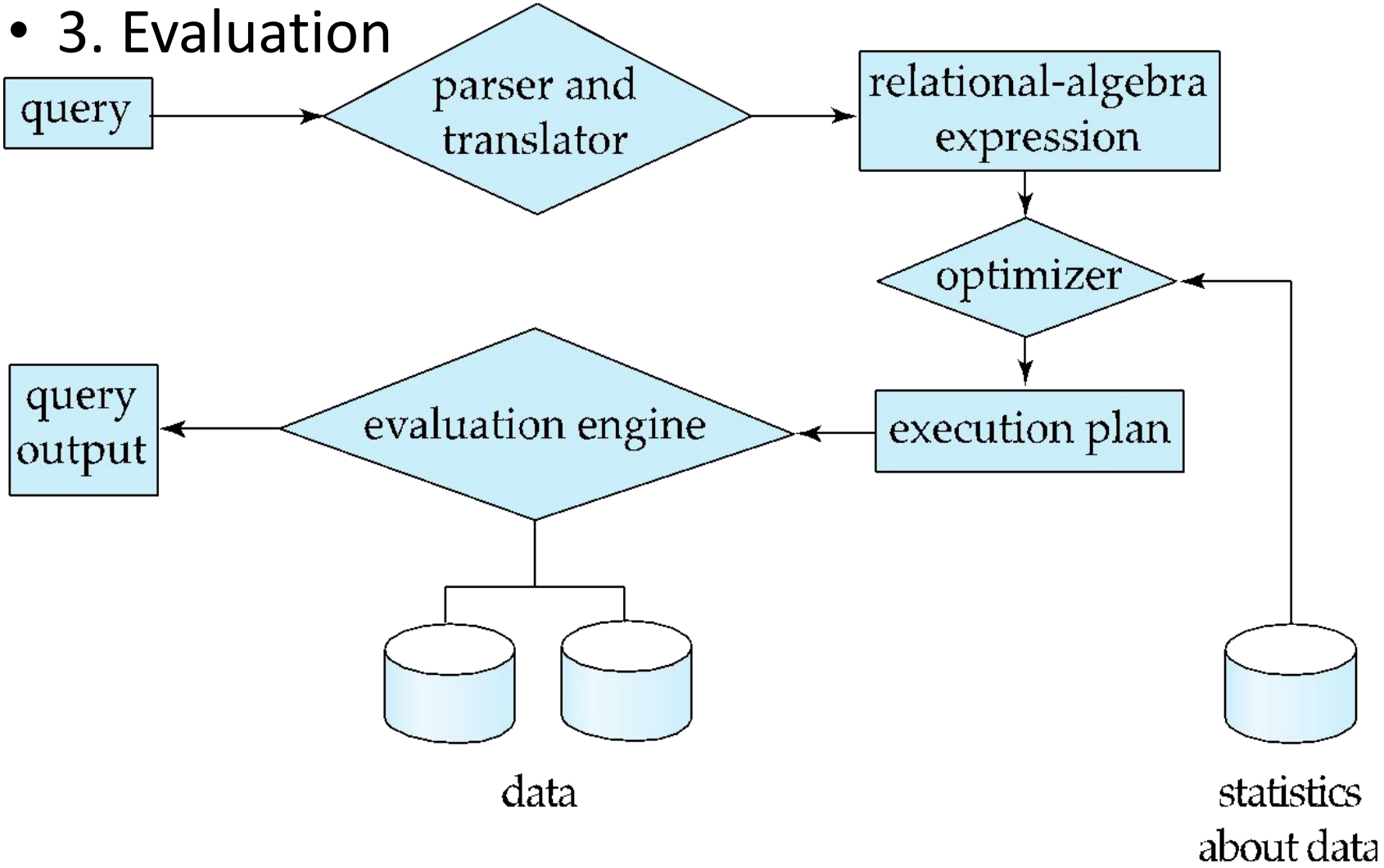
Mapping of N-ary Relationship Types





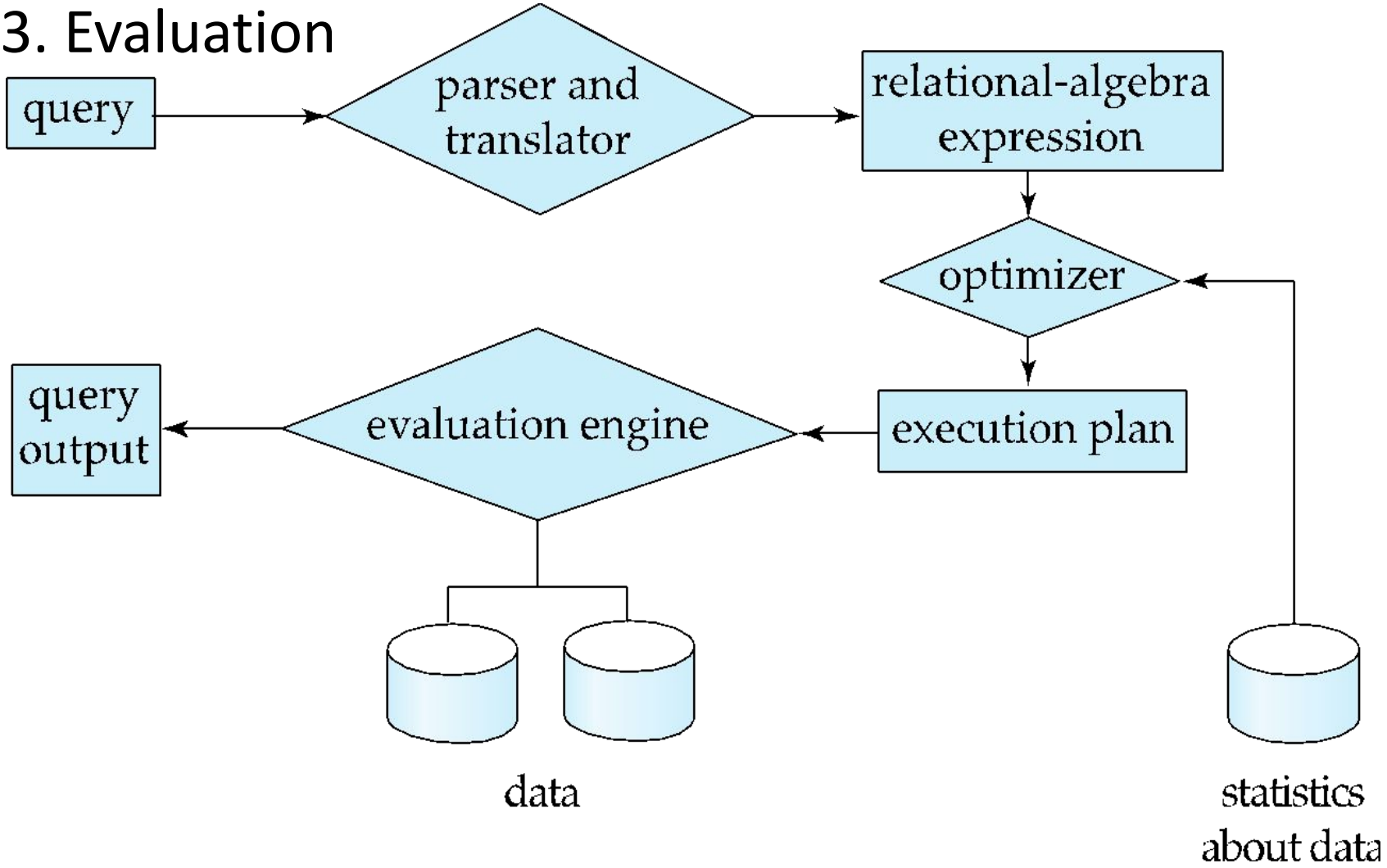
Basic Steps in Query Processing

- 1. Parsing and translation
- 2. Optimization
- 3. Evaluation



Basic Steps in Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



Basic Steps in Query Processing(Cont.)

- Parsing and translation
  - translate the query into its internal form. This is then translated into relational algebra.
  - Parser checks syntax, verifies relations

- Evaluation
  - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.

Basic Steps in Query Processing: Optimization

- A relational algebra expression may have many equivalent expressions
  - E.g., $\sigma_{salary<75000}(\prod_{salary}(instructor))$ is equivalent to $\prod_{salary}(\sigma_{salary<75000}(instructor))$
- Each relational algebra operation can be evaluated using one of several different algorithms
  - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**. E.g.,:
  - Use an index on *salary* to find instructors with salary < 75000,
  - Or perform complete relation scan and discard instructors with salary ≥ 75000

Basic Steps in Query Processing(Cont.)

- **Query Optimization**: Amongst all equivalent evaluation plans choose the one with lowest cost.
  - Cost is estimated using statistical information from the database catalog
    - e.g. number of tuples in each relation, size of tuples, etc.

Measures of Query cost

- Cost is generally measured as total elapsed time for answering query
  - Many factors contribute to time cost
    - *disk accesses, CPU*, or even network *communication*

- Typically disk access is the predominant cost, and is also relatively easy to estimate. Measured by taking into account
  - Number of seeks * average-seek-cost
  - Number of blocks read * average-block-read-cost
  - Number of blocks written * average-block-write-cost
    - Cost to write a block is greater than cost to read a block
      - data is read back after being written to ensure that the write was successful

Measures of Query cost

- For simplicity we just use the **number of block transfers** *from disk and the* **number of seeks** as the cost measures
  - t_T – time to transfer one block
  - t_S – time for one seek
  - Cost for b block transfers plus S seeks
    $b * t_T + S * t_S$
- We ignore CPU costs for simplicity
  - Real systems do take CPU cost into account
- We do not include cost to writing output to disk in our cost formulae

Measures of Query cost

- Several algorithms can reduce disk IO by using extra buffer space
  - Amount of real memory available to buffer depends on other concurrent queries and OS processes, known only during execution
    - We often use worst case estimates, assuming only the minimum amount of memory needed for the operation is available
- Required data may be buffer resident already, avoiding disk I/O
  - But hard to take into account for cost estimation

Selection Operation

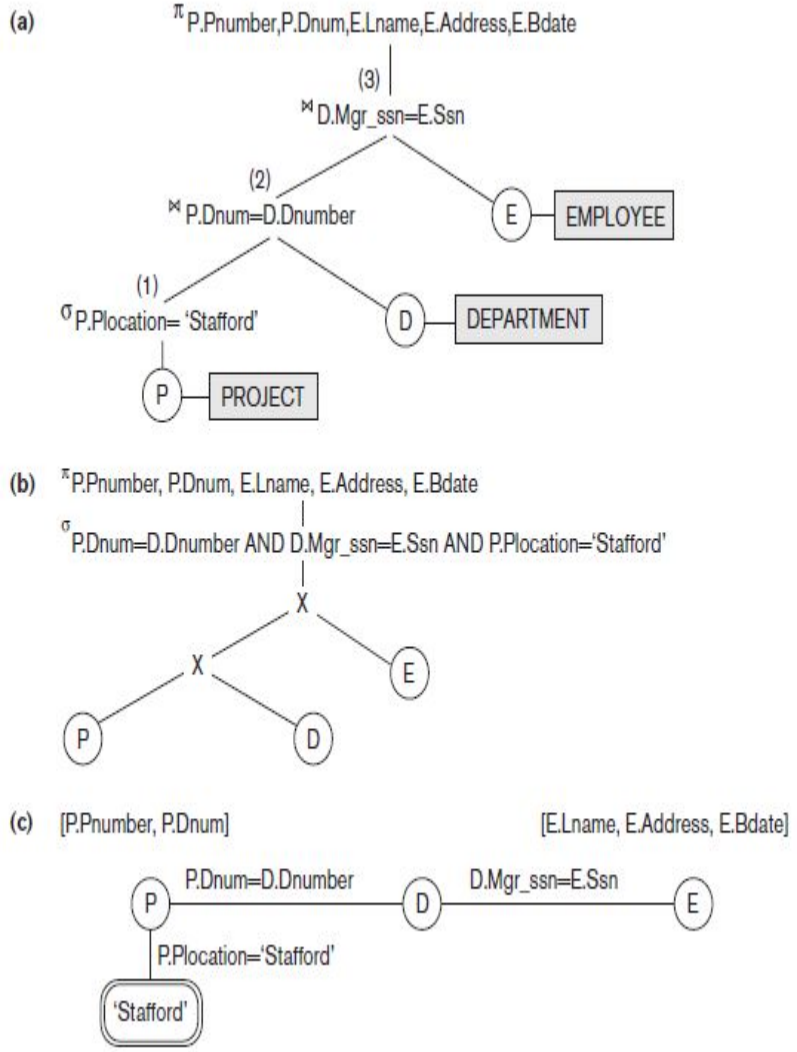
- **Old-A2** *(binary search).* Applicable if selection is an equality comparison on the attribute on which file is ordered.
  - Assume that the blocks of a relation are stored contiguously
  - Cost estimate (number of disk blocks to be scanned):
    - cost of locating the first tuple by a binary search on the blocks
      - $\lceil \log_2(b_r) \rceil * (t_T + t_S)$
    - If there are multiple records satisfying selection
      - *Add transfer cost of the* number of blocks containing records that satisfy selection condition

Query Trees and Heuristics for Query Optimization (cont'd.)

- Example heuristic rule
  - Apply SELECT and PROJECT before JOIN
    - Reduces size of files to be joined
- Query tree
  - Represents relational algebra expression
- Query graph
  - Represents relational calculus expression

Query Trees and Query Graph Corresponding to Q2

- Figure 19.1 Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2.
(b) Initial (canonical) query tree for SQL query Q2.
(c) Query graph for Q2.
-



Query Trees and Heuristics for Query Optimization (cont'd.)

- Query tree represents a specific order of operations for executing a query
  - Preferred to query graph for this reason
- Query graph
  - Relation nodes displayed as single circles
  - Constants represented by constant nodes
    - Double circles or ovals
  - Selection or join conditions represented as edges
  - Attributes to be retrieved displayed in square brackets